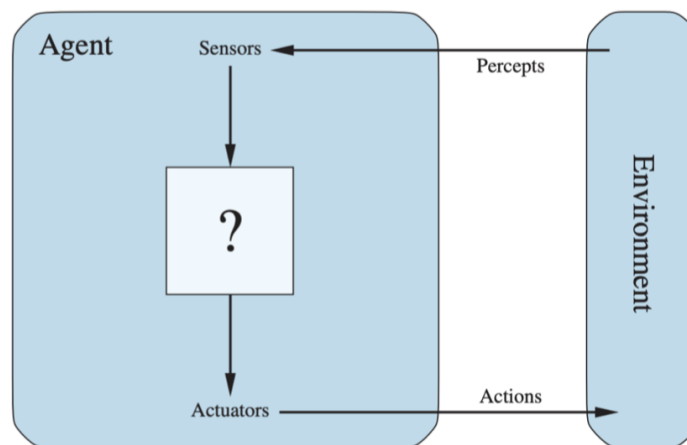


CS 211: Introduction to AI

Lecture 1: Agents

Agents and Environment

1. 智能体 Agent 如何与环境交互: An agent perceives its environment through sensors and acts upon that environment through actuators.
 - a. 环境 Environment: 智能体 Agent 所处的世界。
 - b. 传感器 Sensor: 智能体 Agent 的一部分, is affected by the environment and sends the state into the computations as a value.
 - c. 执行器 Actuator: 智能体 Agent 的一部分, whose state can be set by the agent's computations and affects the environment.
2. 智能体 Agent 如何做出决策: 智能体 Agent 在任何时刻的行为决策取决于其内建知识 built-in knowledge 和完整感知序列 the entire percept sequence observed to date. Agent 不能基于未感知到的信息做决策。
 - a. 感知 Percept: 传感器 Sensor 接收的内容 the content an agent's sensors are perceiving.
 - b. 感知序列 Percept Sequence: 所有的感知 Percept 历史 the complete history of everything the agent has ever perceived.
 - c. 智能体函数 Agent Function: 定义了 Agent 针对 every possible percept sequence 所采取的行动。它是一个数学映射, 描述了智能体如何根据过去的感知做出决策。
 - d. 智能体程序 Agent Program: produces agent function. 它运行在智能体的计算架构上, 接收传感器的输入, 并生成智能体的行为输出。
3. 智能体 Agent 的全流程



4. 案例 Example
 - a. 微波炉 Microwave Ovens

- i. **Sensors:** 温度传感器（检测食物的温度）、湿度传感器（判断食物是否加热完成）、门开关传感器（检测微波炉门是否关闭）、触摸或按钮输入（用户选择加热时间和模式）。
 - ii. **Actuators:** 微波发生器（用于加热食物）、转盘马达（旋转食物以均匀加热）、显示屏（显示时间、模式等信息）、声音警报（完成加热后发出提示音）。
 - iii. **Environments:** 微波炉内部（放置食物的空间）、周围的厨房环境（可能受外界干扰，如开门、停电等）、用户交互（用户决定食物的加热时间和模式）。
- b. 国际象棋 Chess AI
 - i. **Sensors:** 读取棋盘状态（检测棋子的位置）
 - ii. **Actuators:** 计算最佳走法、在棋盘上执行走棋动作
 - iii. **Environments:** single agent environment（AI 对战人类时）、multi agent environment（AI 与 AI 对战）、完全可观测环境（整个棋盘状态对双方都可见）

Good Behavior: The Concept of Rationality

5. 理性智能体 Rational Agent

- a. **Rational Agent** 是做正确事情 right thing 的 agent。其行为的合理性由 consequences 评估。
- b. 性能度量 Performance measure: 用于评估智能体在不同环境下的表现。A performance measure evaluates any given sequence of environment states. 关注行动后的状态，而非行动本身。
- c. 智能体的理性行为是基于对环境最终状态的影响来评估的，而不是具体的行动方式。这意味着：你不需要关注机器人是如何完成清洁的（例如，它是用吸尘模式、拖地模式，还是随机移动）。你只需要关注最终的清洁效果，比如房间是否变干净、覆盖率如何。

6. 四个决定理性智能体的因素 Four factors that determine a rational agent at a given time

- a. 性能度量 Performance measure: defines the criterion of success
- b. 智能体对环境的先验知识 Prior Knowledge of the Environment: 智能体在执行任务前，对环境已有的了解。例如，一个自动驾驶汽车需要知道交通规则。
- c. 智能体可执行的行动 Actions that the Agent Perform: 智能体能够执行的操作集合，例如移动、吸尘、抓取等。
- d. 智能体的感知序列 Percept Sequence to Date: 智能体迄今为止收到的所有感知输入，决定了它对环境的理解。

7. 理性智能体的标准: For each possible sequence, a rational agent should select an action that is **expected to maximize its performance measure**, given the **evidence provided by the percept sequence** and whatever **built-in knowledge the agent has**.
 - a. 智能体应该基于**其感知序列和已有知识选择能够最大化性能度量的行动**。
 - b. 换句话说, 智能体不是随机行动, 而是根据已有信息做出最佳决策。
8. 案例: 吸尘机器人如何更 Rational?
 - a. 核心问题是: 如何让吸尘机器人更“聪明”地清洁, 而不是机械地执行规则? 我们可以增加更强的传感器 (如灰尘检测、电量监测、环境扫描), 优化行动决策 (如路径规划、吸力调整、自我学习)。
 - b. **Improved Sensors:** 灰尘传感器, 能够检测地面灰尘浓度, 而不是仅仅判断“脏”或“干净”。这样机器人可以优先清洁更脏的区域。障碍物传感器, 识别家具或墙壁, 避免碰撞, 提高导航效率。
9. 理性和全知 Rationality and Omniscience
 - a. 理性的智能体并不需要知道所有未来可能发生的事情才能做出决策。
Rationality does not require Omniscience.
 - b. Omniscience is impossible. 在现实世界中, 没有智能体能够完全掌握所有信息, 因此理性智能体只能基于已知信息进行最佳决策。
 - c. 全知智能体 Omniscience Agent: An omniscient agent knows the actual outcome of its actions and can act accordingly. 知道自己所有行动的确切结果, 因此可以完美行动
 - d. Rationality maximizes expected performance (基于当前信息做出最优决策), while perfection maximizes actual performance (基于未来所有可能的结果做出最优决策, 但这不现实)。
10. 信息收集与自主性 Information Gathering and Autonomy
 - a. **Information Gathering:** Agent 可以通过执行某些行动来探索环境, 从而获得更多信息。这些信息可以帮助 agent 优化 future percepts。例如, 一个机器人可以主动移动, 以便感知更多的环境信息, 而不是被动等待信息输入。
 - b. **Autonomy:** 一个理性的 agent 应该是自主的 autonomous, 而不是依赖外部输入做决策。这意味着 agent 应该能够从环境中学习, 而不是完全依赖预先设置的规则。Learn as much as possible **from what it perceives and gather information**.

The Nature of the Environment

11. 智能体的 PEAS 框架

- a. **Performance Measure is a Trade-off:** Performance Measure 决定了智能体的目标, 但不同目标之间可能存在权衡 Trade-off。例如, 准确到达目的地、最

小化燃油消耗、行程时间、成本、交通法规、最大化安全性、旅客舒适度、利润等等。

- b. **Environment Varies:** 智能体运行的环境并不固定，而是根据不同场景变化，比如城市驾驶、郊区驾驶和乡村驾驶。不同的环境会影响智能体的决策和行动策略。
- c. **Actuators and Sensors Depend on Environment and Performance Measure**
 - i. 在自动驾驶系统中，城市驾驶可能需要精细的摄像头、雷达和更先进的避障系统。乡村驾驶可能需要更强大的悬挂系统来适应崎岖地形。
 - ii. 在吸尘机器人中，大面积房屋需要更强的电池和高效的路径规划。小型公寓可能只需简单的传感器和随机移动模式。

12. 智能体的任务环境 **Task Environment:** Task environment 是 agent 运行并尝试实现目标的环境。为了构建一个 agent，首先要尽可能详细地定义其任务环境，确保它能够在该环境中合理运作。

13. 任务环境的七个维度 **Seven Dimensions of the Task Environment**

- a. 完全可观测/部分可观测 **Fully Observable/Partially Observable**
 - i. 完全可观测 **Fully Observable:** Agent 能够获取环境的所有相关信息。例如，国际象棋游戏，棋盘上的所有信息都对智能体可见。传感器可以检测环境中所有与决策相关的信息，使智能体可以直接采取最优行动。智能体不需要维护内部状态，因为所有必要的信息都可以从传感器获取。
 - ii. 部分可观测 **Partially Observable:** Agent 只能获得部分环境信息，需要推断缺失的部分。例如，自动驾驶汽车可能因天气或障碍物无法看到所有路况。传感器只能获取部分信息，智能体需要**依赖内部状态或记忆**来进行决策。
 - iii. 不可观测 **Unobservable:** 如果智能体没有传感器，那么环境对它来说就是完全不可观测的。这意味着智能体无法感知环境状态，也就无法做出合理的决策。
- b. 单智能体/多智能体 **Single Agent/Multi Agent**
 - i. 单智能体 **Single Agent:** 环境中只有一个 Agent 进行决策，如普通的自动化吸尘器。
 - ii. 多智能体 **Multi Agent:** 多个 Agent 相互影响，如自动驾驶车辆需要考虑其他车辆、行人和交通规则。
- c. 确定性/非确定性 **Deterministic/Nondeterministic**
 - i. 确定性：在完全可观测的情况下，智能体可以理性地选择最优动作，以最大化其性能评估标准。给定当前状态和行动，下一状态是唯一确定的。例如，传统棋类游戏（国际象棋）。

- ii. 非确定性：相同的动作可能导致不同的结果，通常由于环境的不可控因素。例如，自动驾驶中的突发天气变化。如果这种不确定性可以被量化（如概率分布），那么它就是一个随机环境（Stochastic Environment）。
- d. 阶段性/顺序性 Episodic/Sequential
 - i. 阶段性任务 Episodic：每个决策都是独立的，不影响未来状态。例如，垃圾分类机器人只需识别当前物体，不关心之前的物体。
 - ii. 顺序性任务 Sequential：当前决策会影响未来状态（有序的 in series）。例如，国际象棋中当前的走子会影响后续决策。
- e. 静态/动态 Static /Dynamic
 - i. 静态环境 Static：环境不会随时间变化，只有智能体的行动会改变环境状态。例如，填字游戏。
 - ii. 动态环境 Dynamic：环境会随时间变化，即使智能体不采取行动。例如，自动驾驶中的道路交通情况。
 - iii. 半动态环境 Semi-Dynamic Environment：环境本身不变化，但智能体的 Performance Score 会随着时间改变。适用于那些智能体的行为虽然不直接改变环境，但**拖延或无作为会影响最终得分**。比如考试系统，时间流逝，但试卷内容不变，若考生拖延，最终得分会降低。
- f. 离散/连续 Discrete/Continuous
 - i. 离散环境 Discrete：状态、感知和动作都是离散的，如国际象棋，每个棋子有固定的移动方式。
 - ii. 连续环境 Continuous：状态和动作是连续的，如自动驾驶，转向角度、加速、减速都是连续变量。
- g. 已知/未知 Known/Unknown
 - i. 已知环境 Known：智能体完全了解环境的规则，例如象棋 AI 已知棋盘规则。
 - ii. 未知环境 Unknown：智能体需要探索和学习环境，例如无人机在未知地形中导航。
 - iii. **已知环境不等于可完全观测**：即使一个环境的规则已知，智能体也不一定能观测到环境的全部状态。比如纸牌游戏，玩家可能知道游戏规则，但不一定能看到对手的手牌（即环境是部分可观测的）。
 - iv. **未知环境不等于不可观测**：即使环境的规则未知，智能体仍然可能能够观测到环境的部分或全部状态。比如视频游戏，玩家可能不清楚游戏机制（未知环境），但可以通过屏幕看到游戏世界（可观测环境）。

The Structure of Agents

14. 智能体程序与架构 Agent Program and Architecture

- a. 智能体程序 Agent Program: Agent program 是构建 rational agent 的关键，其作用在于实现 agent function，即将 Percepts 映射到 Actions。
- b. 智能体架构 Agent Architecture: Agent program 运行在 Agent architecture 上。Agent architecture 通常是一个计算设备 computing device，配备 sensors 和 actuators，用于感知环境并采取行动。
- c. Agent = architecture + program，这意味着 agent 由硬件（架构）和软件（程序）共同构成。

15. 智能体程序是函数的具体实现

- a. Agent program 是 agent function 的具体实现，agent function 是一个抽象概念，其基于 entire percept history 做出决定，其输入是 entire percept history，作用是依赖完整的过去信息进行决策。而 agent program 仅基于 current percept，其在物理硬件上运行。在实际应用中，Agent Program 可能会使用更高效的方式来逼近 Agent Function，比如仅基于当前感知。
- b. Agent program 和 agent function 的对比如下

Agent program	Agent function
是 agent function 的具体实现	一个数学映射，定义了从感知输入到行动输出映射规律 $f: Percept History \rightarrow Action$
在物理硬件，也就是 Agent architecture 上运行	是一个抽象概念
takes the current percept as input	the entire percept history
Agent Program 是 Agent Function 的实现方式，它运行在物理架构上，计算并生成符合 Agent Function 预期的行为。在实际应用中，Agent Program 可能会使用更高效的方式来逼近 Agent Function，比如仅基于当前感知。	

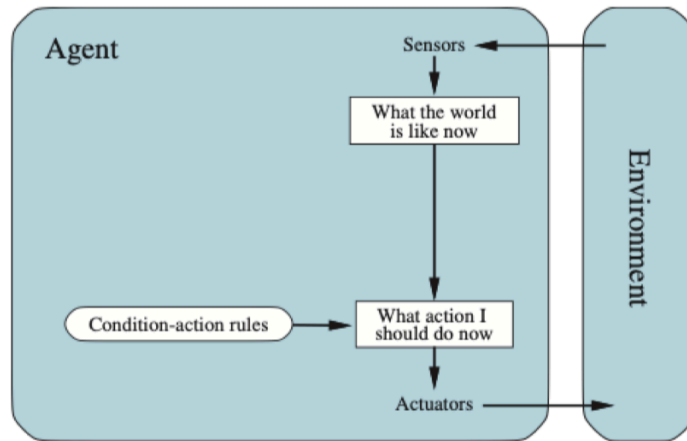
16. 智能体的不同类型 Different Types of Agent

- a. 简单反射型智能体 Simple Reflex Agents: 只根据当前 percept 做出决策，而不考虑过去的感知或世界状态。例子：基于条件-动作规则（if-then rules）的智能体，例如简单的自动门。
- b. 基于模型的反射型智能体 Model-based Reflex Agents: 维护一个内部状态（Model），用来表示环境的历史信息，从而做出更复杂的决策。例子：智能温控器，它不仅感知当前温度，还记住最近的温度变化趋势。
- c. 基于目标的智能体 Goal-based Agents: 具有明确的目标（Goal），智能体在做决策时会考虑如何朝着目标迈进。例子：路径规划算法，如 A* 算法，它基于目标状态（目的地）来选择行动。

- d. 基于效用的智能体 **Utility-based Agents**: 除了目标之外, 还使用 **Utility Function** 来衡量每种可能状态的“好坏”程度, 从而选择最优策略。例子: 自动驾驶汽车, 它不仅考虑如何到达目的地, 还会评估行驶时间、油耗、安全性等多个因素。
- e. 学习型智能体 **Learning Agents**: 通过与环境交互不断学习和优化决策, 通常包含 **Learning Component**。例子: 强化学习 (**Reinforcement Learning**) 算法, 如 AlphaGo。

17. 简单反射型智能体 Simple Reflex Agents

- a. The simplest agent: Actions are relying based on the current percept. The percept history is ignored
- b. 该智能体不会存储或参考过去的 **percept**, 因此无法适应长期变化的环境。例如, 光敏机器人只能根据当前光照强度决定行动, 而不会记住之前的光照条件。



- c. Pseudocode implementation: 这个 Simple Reflex Agent 仅依据当前的 **percept** 作出决策, 而不存储过去的 **percept history**, 因此它只能适用于 **fully observable environment** 中工作。如果环境是 **partially observable**, 它就可能无法确定正确的行动。

```

function SIMPLE-REFLEX-AGENT(percept) returns an action
    persistent: rules, a set of condition–action rules

    state ← INTERPRET-INPUT(percept)
    rule ← RULE-MATCH(state, rules)
    action ← rule.ACTION
    return action

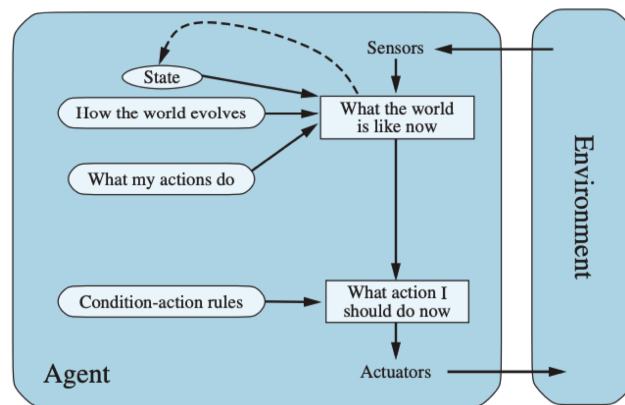
```

- d. Deterministic Simple Reflex Agents and Randomized Simple Reflex Agents
 - i. Deterministic Simple Reflex Agents 总是对相同的感知做出相同的决策。

- ii. Randomized Simple Reflex Agents 可能在某些情况下表现更好，比如：避免陷入循环错误决策（如在迷宫中重复相同动作）。

18. 基于模型的反射型智能体 Model-Based Reflex Agents

- a. Keeps track of the part of the world it cannot see now (unobserved or partially observed environment). Maintains some **internal state** that depends on the **percept history** and thereby reflects **at least some of the unobserved aspects of the current state**.
- b. 该智能体会存储 **percept history**，以便在做决策时考虑之前的信息。适用于部分可观察环境（部分信息无法直接通过感知获取）。
- c. Requires two knowledge
 - i. Transition model: 获取 how the world changes over time 的信息。这包括了 agent 的 action 如何影响环境以及环境如何独立 evolution。
 - ii. Sensor model: 获取 the state of the world 如何映射到 agent 的 percepts。智能体通过 sensor 感知环境的方式由该模型决定。



- d. Pseudocode implementation: 相比于 Simple Reflex Agents, Model-Based Reflex Agent 维护内部状态，可以在部分可观察环境下运行。通过 transition model 和 sensor model，智能体可以推理环境状态，而不仅仅依赖当前感知。更具适应性，能处理更复杂的决策环境。

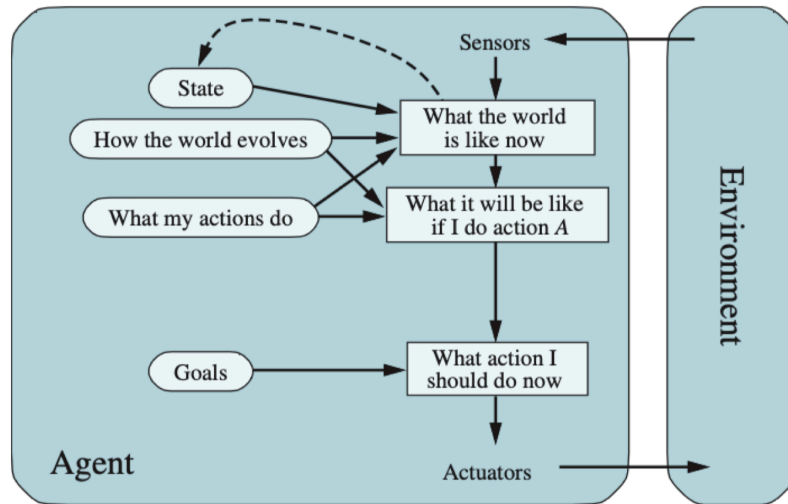
```

function MODEL-BASED-REFLEX-AGENT(percept) returns an action
  persistent: state, the agent's current conception of the world state
               transition_model, a description of how the next state depends on
                 the current state and action
               sensor_model, a description of how the current world state is reflected
                 in the agent's percepts
               rules, a set of condition-action rules
               action, the most recent action, initially none

  state ← UPDATE-STATE(state, action, percept, transition_model, sensor_model)
  rule ← RULE-MATCH(state, rules)
  action ← rule.ACTION
  return action
  
```

19. 基于目标的智能体 Goal-Based Agents

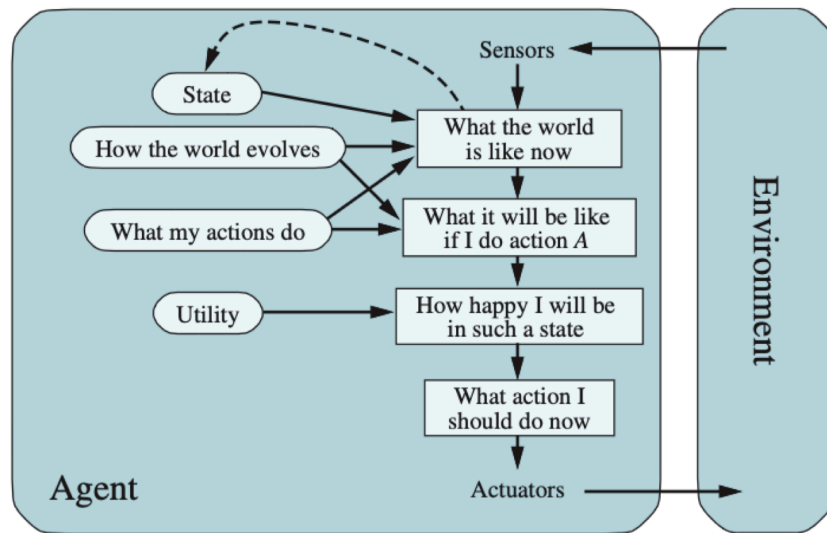
- a. Knowledge about the current state of the environment is not always enough to decide what to do. An agent needs some goal information that describes desirable situations.
- b. 该 agent 会 search 和 planning 一系列行动序列，以实现既定目标。这与 simple reflex agents 不同，goal-based agents 会考虑长期规划。不过，这种方法可能 less efficient，因为需要搜索和计算，但是它更 flexible。其决策支持知识是显式存储的，可以根据环境变化进行修改。



20. 基于效用的智能体 Utility-Based Agents

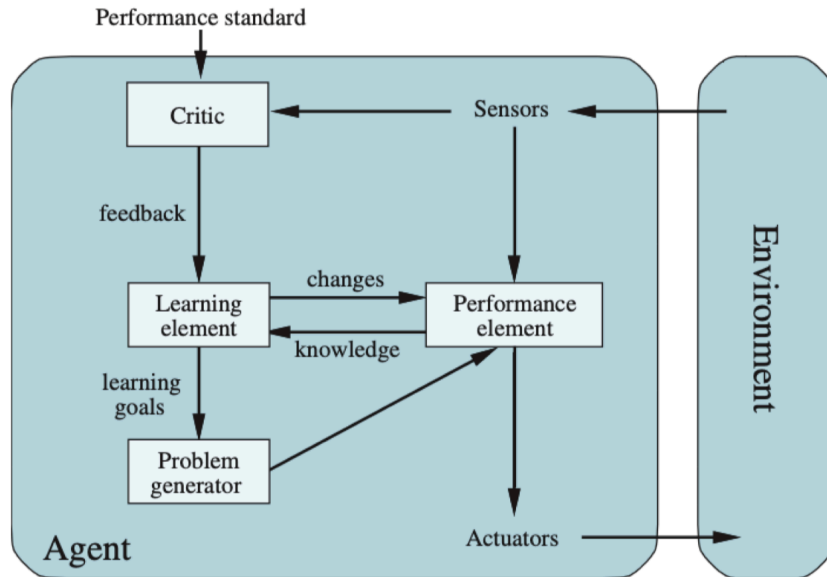
- a. 仅仅设定目标可能无法在复杂环境中生成最佳行为。目标提供的是 binary 区分，即目标只提供“成功/失败”的粗略评判，而不考虑不同选择的质量。Utility 代表了 quality of being useful，不仅仅是是否达成目标，而是衡量不同可能结果的优劣。
- b. A comparison of different world states according to utility can make an agent more rational.
- c. Utility Function
 - i. The utility function is an internalization of the performance measure.
 - ii. 效用函数和外部性能度量的一致性 agreement: 智能体通过 maximize utility 来提高理性程度，从而在决策时更合理。
 - iii. Utility-based agents 比单纯的 goal-based agents 具备更强的适应性，可以更灵活地学习并优化决策。
 - iv. 当多个目标发生冲突时，utility function 可以 tradeoff，帮助 agent 选择最优方案。
 - v. Agent 可以综合考虑 the likelihood of success 和 the importance of the goals，而不仅仅是简单的“目标是否达成”。

- d. 决策是在 uncertainty 下进行的
 - i. 现实环境往往充满不确定性，agent 需要在不确定性下做出最优决策。
 - ii. Agent 图基于每个可能结果的效用和概率，计算 expected utility，选择最优行动。这一点类似于期望收益最大化的决策理论（如经济学和统计学中的期望值计算）。
- e. Global Rationality/Local Rationality: 在复杂环境中，智能体无法计算所有可能的全局最优解，所以它会采用局部最优 local rationality，即基于当前信息做出最佳决策。

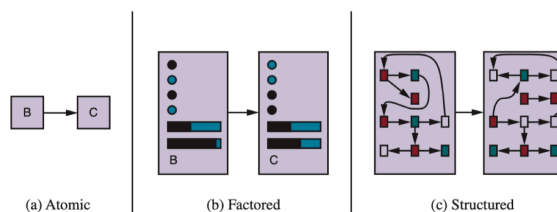


21. 学习型智能体 Learning Agents

- a. Learning allows the agent to operate in initially unknown environments and become more competent.
- b. Learning 使 agent 能够在初始未知的环境中运作，并随着时间推移不断提高自身能力和决策水平。这种类型的智能体在探索和适应复杂或动态环境时尤为重要，如 Reinforcement Learning 或自适应 AI 系统。
- c. 四个组成部分
 - i. **Learning element**: 负责改进 agent 的表现。通过经验积累与优化，调整 agent 的内部模型或策略，使其在未来的决策中表现更好。
 - ii. **Performance element**: 负责 selecting external actions。基于当前的 percept 来决定采取 actions。
 - iii. **Critic**: 负责 sending feedback to the learning element。不直接影响 actions 选择，但是帮助 agent 理解哪些决策是好的，哪些需要改进。
 - iv. **Problem Generator**: 负责 suggesting actions that will lead to new and informative experiences。



- d. Learning is a process of modification of each component of the agent to bring the components into closer agreement with the available feedback information, thereby improving the overall performance of the agent.
- e. **State Representation:** Agent 在学习过程中使用的状态表示方式 State Representation, 主要包括三种不同的表示方法。
 - i. **Atomic Representation:** 视状态为一个整体的黑箱, 没有内部结构。不能访问或操作状态的子部分, 智能体只能基于整个状态做决策。适用于最简单的决策模型, 例如某些基于查找表 (Lookup Table) 的强化学习方法。示例: 国际象棋的一个棋盘状态被编码为一个单一的索引值, 不能进一步分解。
 - ii. **Factored Representation:** 将状态拆分为多个变量或属性, 每个属性都有一个值 (如布尔值、实数、固定向量等)。允许智能体在决策时考虑不同的状态特征, 而不仅仅是整个状态。在自动驾驶中, 一个状态可以由多个变量组成, 例如: 位置 (GPS 坐标)、速度 (km/h)、交通灯状态 (红/绿/黄)、周围车辆距离 (数值)。
 - iii. **Structured Representation:** 状态不仅包含属性, 还包含对象及其关系, 形成更复杂的表示。在机器人任务规划中, 一个房间的状态可以表示为: 对象 (桌子、椅子、杯子)、关系 (杯子在桌子上、椅子靠近桌子)。



f. 信息表示 Representation

- i. 局部表示 Localist Representation: 每个概念都对应一个单独的存储单元（如神经元或内存位置），并且该存储单元唯一地激活以表示该概念。one-to-one mapping between concepts and memory locations
- ii. 分布式表示 Distributed Representation: 分布式表示是指一个概念由多个存储单元（如多个神经元）共同编码，并且每个存储单元可以同时用于多个概念的代表。the representation of a concept is spread over many memory locations, and each memory location is employed as part of the representation of multiple different concepts
- iii. Distributed representations are more robust against noise and information loss.

Lecture 2: Uninformed Search

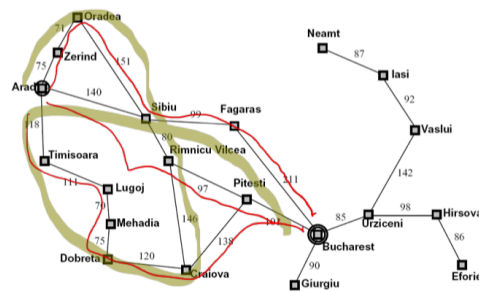
Agents that Plan Ahead

1. 反射型智能体与规划型智能体 Reflex and Planning Agents

- a. **Reflex agent:** 根据 current percept 选择行动，可能会使用一些记忆。可能具有部分记忆或世界状态的模型。不考虑未来行为的后果，只 consider how the world IS（仅依赖当前 percept）。
- b. **Planning agent:** 提出 what if 问题，即考虑不同行动的假设后果，而不是仅仅根据当前的 percept 做决定。Must have a model of how the world evolves in response to actions and must formulate a goal. Consider how the world WOULD BE.
 - i. **Optimal planning:** 在已知环境下，寻找最佳的行动序列以最大化目标达成率。
 - ii. **Complete planning:** 确保所有可能的情况都被考虑，并找到适用于所有可能性的行动序列。
 - iii. **Planning:** 智能体在执行任务之前做出完整的计划。
 - iv. **Replanning:** 如果环境发生变化，智能体能够调整计划并适应新的情况。

2. Search Problems

- a. A search problem consists of:
 - i. **State space:** 定义了一组可能的状态，agent 可以在这些状态直接转换。比如吃豆人的游戏场景，每个 state 表示吃豆人在某个位置，周围有不同的路径和目标。
 - ii. **Successor function:** 定义从一个 state 如何转换到另一个 state 的规则。包括
 - **Actions:** 如向北移动 N 或向东移动 E。
 - **Costs:** 每个动作的代价，比如吃掉几个豆，或者走了几步。
 - iii. **Start state and goal test**
 - Search problem 需要一个明确的 start state（最初的位置）
 - Goal test 用于判断是否到达目标状态，即找到问题的 solution。
- b. **Solution:** 一个 solution 是一个 action sequence，即一个 plan，能够将 agent 从 start state 转化到 goal state。目标是找到最优路径，即 action sequence 的 cost 最低，并成功达到 goal state。
- c. **Example:** Traveling in Romania



- State space:
 - Cities
- Successor function:
 - Roads: Go to adjacent city with cost = distance
- Start state:
 - Arad
- Goal test:
 - Is state == Bucharest?
- Solution?

3. World State vs. Search State

- a. **World state:** 包含环境中的所有细节，比如障碍物，敌人，食物点等等。适用于全面描述世界，但不总用于规划（因为可能包括太多无关信息）。比如，吃豆人的游戏节目，包括迷宫、幽灵、豆子、吃豆人的位置等等。The world state includes every last detail of the environment.
- b. **Search state:** 只保留对 plan 有用的信息，省略无用细节，让 search 更搞笑。通过 abstraction 来表示状态。比如在路径规划中，我们只关心吃豆人的位置，而不考虑幽灵的位置。A search state keeps only the details needed for planning (abstraction)。

4. 两种搜索问题 Pathing/Eat-All-Dots

- a. **Problem: Pathing**
 - i. **States:** 仅考虑吃豆人的(x,y)坐标位置。
 - ii. **Actions:** 向东南西北移动。
 - iii. **Successor:** 每个动作都让吃豆人移动到新的(x,y)位置。
 - iv. **Goal test:** 检查吃豆人有没有到达目标位置(x,y)=END。
 - v. **特点:** 这个问题是寻找最短路径，只关心如何从起点移动到终点。
- b. **Problem: Eat-All-Dots**
 - i. **States:** 不仅考虑吃豆人的坐标，也记录哪些豆子被吃了（使用 Boolean 表示）
 - ii. **Actions:** 向东南西北移动。
 - iii. **Successor:** 吃豆人移动到新的(x,y)位置；如果移动到有豆子的地方，豆子的 Boolean 变为 false。
 - iv. **Goal test:** 检查是否所有豆子都被吃掉（所有 dot booleans 变为 false）。
 - v. **特点:** 这个问题比路径规划复杂，因为它涉及多个目标（所有豆子）。Pac-Man 需要制定策略来最优地吃掉所有豆子，而不是仅仅找到一条路径。

c. Example

- World state:
 - Agent positions: 120
 - Food count: 30
 - Ghost positions: 12
 - Agent facing: NSEW
- How many
 - World states? $120 \times (2^{30}) \times (12^2) \times 4$

Handwritten notes: T/F of all foods, ghost, directions
 - States for pathing? 120
 - States for eat-all-dots? $120 \times (2^{30})$

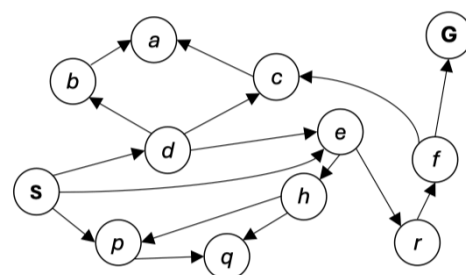
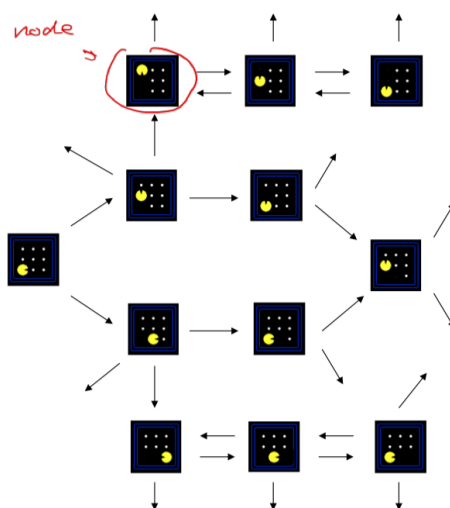
5. State Space Graphs and Search Trees

a. State space graph: Search problem 的抽象数学模型，用于描述不同 state 之间的转换关系。

- Nodes: 每个 Nodes 代表一个(abstraced) world state，即系统的某个 configurations。
- Arcs: 连接不同 Nodes，表示 successors，通常对应某个 actions 及其结果。
- Goal Nodes: State space graph 中存在一个或多个 goal states，搜索算法的目标是找到起点到 goal states 的路径。

b. State space graph 的特点

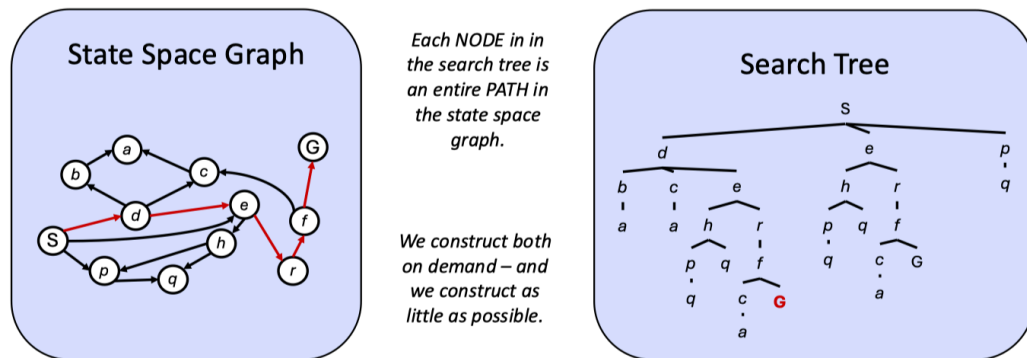
- In a state space graph, each state occurs only once! 即状态空间图是一个有向图，每个唯一状态只有一个节点。
- We can rarely build this full graph in memory (it's too big), but it's a useful idea.



Tiny search graph for a tiny search problem

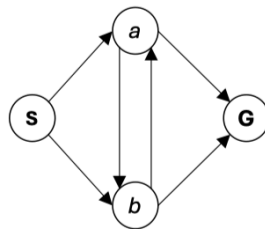
c. Search Trees: What if 树，表示所有可能的计划及其潜在结果。

- i. Root node: search 的起点。
 - ii. Children node: 对应 possible futures, 由当前状态执行某个动作 (Action) 后得到的继承状态
 - iii. Edges: 对应执行的某个 actions。
- d. State Space Graphs and Search Trees
- i. 在 State Space Graph 中, 每个状态只出现一次, 不会重复, 包含整个问题空间, 即所有可能的状态及其相互关系。
 - ii. 在 Search Tree 中, 可能包含重复状态, 因为不同的路径可能会到达相同的状态。搜索算法不会一次性构建整个搜索树, 而是根据需求逐步展开 (构造尽可能少的节点)。
 - iii. Lots of repeated structure in the search tree!



Consider this 4-state graph:

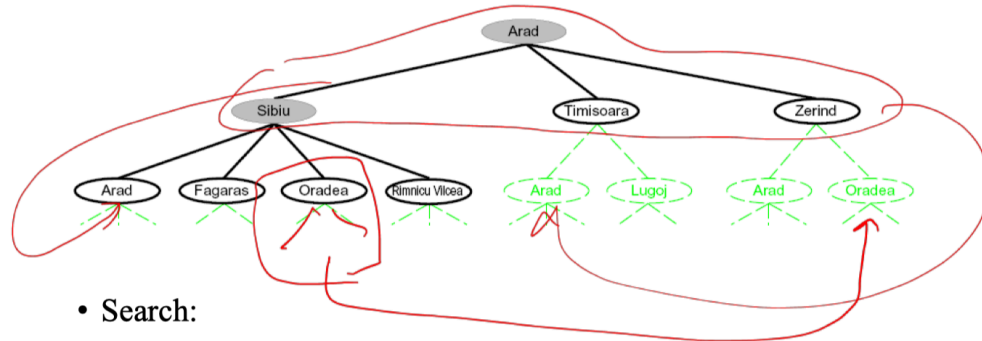
How big is its search tree (from S)?



6. General Tree Search

- a. Fringe: 所有待扩展的 node。搜索策略的核心问题是: 从 fringe 中选择哪一个节点进行扩展? (图中绿色的为 fringe)
- b. Expansion: 选择当前的某个 node, 并展开他的 children node。形成新的 search tree 分支。
- c. Search strategy: 尽量扩展尽可能少的节点, 以减少计算开销, 提高搜索效率。
- d. Exploration strategy: 决定选择哪个 fringe 节点进行扩展。

- i. BFS 广度优先搜索
- ii. DFS 深度优先搜索
- iii. A*搜索



- Search:
 - Expand out potential plans (tree nodes)
 - Maintain a **fringe** of partial plans under consideration
 - Try to expand as few tree nodes as possible

e. Tree search algorithm

```

function TREE-SEARCH( problem, strategy) returns a solution, or failure
  initialize the search tree using the initial state of problem
  loop do
    if there are no candidates for expansion then return failure
    choose a leaf node for expansion according to strategy
    if the node contains a goal state then return the corresponding solution
    else expand the node and add the resulting nodes to the search tree
  end

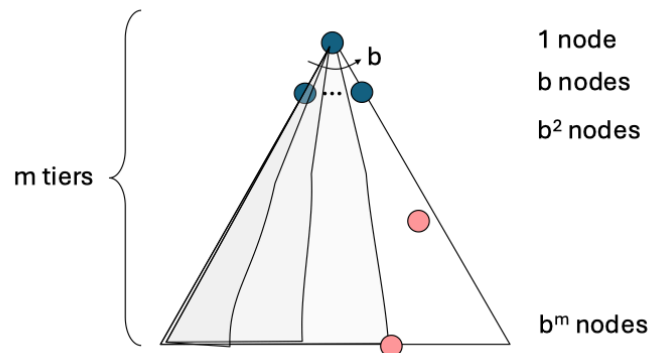
```

- i. 初始化搜索树，起始状态为问题的 initial state。
- ii. 进入 loop: 如果没有可扩展的候选节点，返回失败。
- iii. 根据搜索策略选择一个叶节点进行扩展（fringe 的选择）。
- iv. 如果该节点包含目标状态，返回解决方案。
- v. 否则，扩展该节点，将新生成的子节点加入 search tree。

f. Search Algorithm Properties

- i. Complete: Guaranteed to find a solution if one exists?
- ii. Optimal: Guaranteed to find the least cost path?
- iii. Time complexity and space complexity: 取决于搜索算法的技术细节
- iv. Cartoon of search tree
 - b 是 branching factor, 即每个节点的子节点数
 - m 是 maximum depth
 - solutions at various depths

- Number of nodes in entire tree: $1 + b + b^2 + \dots + b^m = O(b^m)$
($O(b^m)$ 是 upper bound, 即所有层的节点总数)



Depth-First Search

7. Depth-First Search 的策略与实现

- Strategy:** 优先扩展最深的 node, 即每次深入 search tree 的最深处, 再回溯到上一级寻找未探索的路径。
- Implementation**
 - LIFO 机制:** 最新加入的 node 会最先被扩展 (即优先探索最新加入的分支)
 - 早加入的 node 在最新 node 被探索完之前不会被扩展。
 - Example:** 假设有如下 state space, initial state S 通过 3 条路径可以到达 A、B、C。
 - 初始化: Fringe 只包含起始节点 S: Fringe: [S]
 - 展开 S, 将其子节点 A, B, C 按顺序入栈 (注意: 顺序不同会影响搜索路径): Fringe: [A, B, C] (C 是最后加入的, 最先被探索)
 - 取出 C 进行扩展 (因为栈是 LIFO, 最后进去的 C 先被取出), 假设 C 有子节点 C1 和 C2: Fringe: [C1, C2, A, B]
 - 继续取出 C1 进行扩展, 直到 C 及其子节点被探索完, 然后回溯到 B 和 A。

8. Depth-First Search (DFS) Properties

- DFS 扩展哪些节点:** 可能会扩展整个搜索树 (如果没有找到解)。DFS 优先沿着树的**最左侧路径** (我们设置最左端为最新加入的) 向下搜索, 直到到达最大深度, 然后才会回溯并探索其他分支。
- Time complexity:** $O(b^m)$ b 是总 node 数;

- c. **Space complexity:** $O(bm)$ 远小于广度优先搜索的 $O(b^s)$ 。这意味着 DFS 空间占用较小, 适用于内存受限的情况。
- d. **Completeness:** 不一定, 如果 m 是无限的 (例如遇到死循环或无限深度的搜索树), DFS 可能永远不会找到解。但是, 如果我们限制搜索深度或防止重复状态 (例如使用循环检测), DFS 可以是完整的。
- e. **Optimality:** 不是最优的, 因为 DFS 总是找到最左侧的解, 而不考虑路径的深度或代价。

Breadth-First Search

9. Breadth-First Search 的策略与实现

- a. **Strategy:** 优先扩展最浅层 (shallowest node first) 的节点, 即先遍历完当前深度的所有节点后, 才进入下一层的节点扩展。
- b. **Implementation:** 使用 FIFO (先进先出) 队列作为 fringe 来存储待探索的节点, 先加入的节点会先被扩展 (与 DFS 的 LIFO 方式不同)
- c. 由于 BFS 采用 逐层扩展 的方式, 它的搜索路径通常是从上到下, 确保最先找到的解是最短路径解。

10. Breadth-First Search (BFS) Properties

- a. **BFS 扩展哪些节点:** 会处理所有浅于最浅解 (shallowest solution) 深度 s 的节点。 **Let depth of shallowest solution be s**
- b. **Time complexity:** $O(b^s)$
- c. **Space complexity:** $O(b^s)$
- d. **Completeness:** 如果 s 是有限的, 即如果解存在并且搜索不会进入无限循环, 则 BFS 一定能找到解。
- e. **Optimality:** 仅当所有路径的 **cost 相等** (例如每条路径的 cost 都是 1) 时, BFS 才能保证最优解。如果路径代价不相等, BFS 不一定找到代价最小的路径。

Depth-First Search 与 Breadth-First Search

11. BFS outperform DFS 的情况

- a. 最优解是最浅的解 (Shallowest Goal)
 - i. BFS 保证找到最浅的解, 而 DFS 可能找到一个更深的非最优解。
 - ii. 适用于路径代价相同的情况下寻找最短路径。
- b. 搜索空间是有限的, 并且解离根节点较近
 - i. BFS 会层层展开, 不会因进入深层递归而浪费时间。
 - ii. DFS 可能会探索不必要的深层路径, 而 BFS 可以更快找到解。
- c. 有环的图, 防止无限递归

- i. DFS 可能会陷入循环（如在没有额外检测时可能无限递归）。
- ii. BFS 由于记录了所有访问的节点，不会陷入死循环。

12. DFS outperform BFS 的情况

- a. 搜索空间非常大，BFS 内存消耗过高
 - i. BFS 需要存储所有的 fringe，其空间复杂度为 $O(b^s)$ ，容易造成内存溢出。
 - ii. DFS 只存储当前路径，空间复杂度 $O(bm)$ ，因此在 存储受限的情况下，DFS 更优。
- b. 目标节点在深层：BFS 需要遍历所有较浅层的节点，DFS 可能会直接进入深层找到解，特别是如果解刚好在 DFS 选择的路径上。

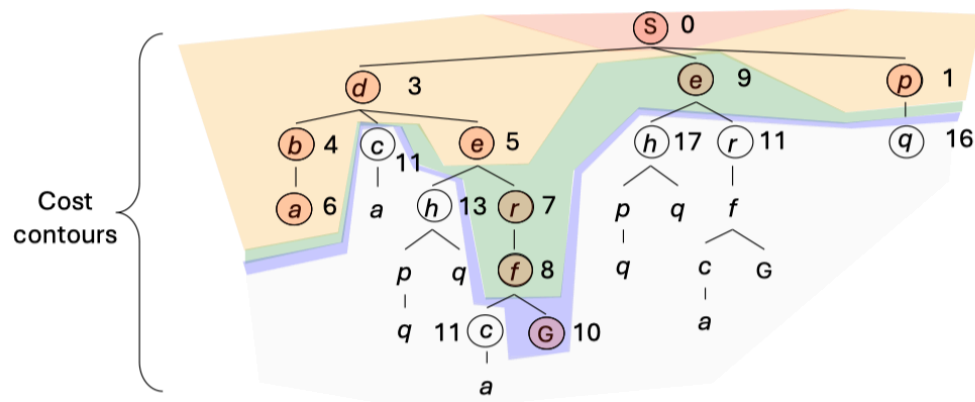
13. Iterative Deepening Search

- a. IDS 结合 DFS 的空间优势 和 BFS 在时间复杂度及找到最浅解上的优势。
- b. 具体方法
 - i. 先运行一个 DFS，深度限制为 1，如果找不到解，则继续……
 - ii. 再运行一个 DFS，深度限制为 2，如果找不到解，则继续……
 - iii. 再运行一个 DFS，深度限制为 3，如此循环，直到找到解。
- c. 实际应用中，IDS 的时间复杂度和 BFS 相同 $O(b^d)$ ，但占用的空间却是 $O(bd)$ ，比 BFS 的 $O(b^d)$ 小很多。
- d. 适用于搜索深度未知，但解可能较浅的情况。

Uniform Cost Search（适用于路径代价不相等的情况）

14. Uniform Cost Search 的策略与实现

- a. Strategy: 扩展累计代价最小的节点（cheapest node first）。节点的优先级由 cumulative cost 决定。
- b. Implementation: 初始状态 S 代价为 0。逐步扩展代价最小的节点（如 p 代价 1、d 代价 3）。形成 等成本扩展区域 cost contours，即相同代价的路径优先级相同，边界上的节点将是下一个扩展的目标。



15. Uniform Cost Search (UCS) Properties

- a. UCS 扩展哪些节点：会扩展所有 cost 小于最优解代价 C^* 的 nodes。我们设最优路径的 cost 是 C^* ，最小 arcs cost 为 ϵ ，则有效深度近似为 C^*/ϵ 。这意味着尽管 UCS 没有明确的深度限制，但是其搜索过程依然收到 C^*/ϵ 约束。
- b. Time complexity: $O(b^{C^*/\epsilon})$
- c. Space complexity: $O(b^{C^*/\epsilon})$
- d. Completeness: YES，如果最优解的 cost 有限，所有边的 cost 都是正数。
- e. Optimality: YES。它总是优先扩展当前代价最小的路径。
- f. 优点与核心特点
 - i. UCS 按照逐步增加的代价轮廓（cost contours）进行搜索，即从最低代价开始，逐渐扩展更高代价的路径。
 - ii. UCS 是完整（Complete）且最优（Optimal）的。
- g. 问题
 - i. Explores options in every “direction”。UCS 不会优先朝向目标方向，而是均匀地向所有可能的方向扩展。这意味着它可能会探索很多不必要的路径。
 - ii. No information about goal location。UCS 仅仅依据路径的累积代价来进行扩展，但它不考虑目标的距离。在某些情况下，UCS 可能会绕远路，因为它无法预先判断哪条路径更接近目标。

Lecture 3: Informed Search

Search Heuristics

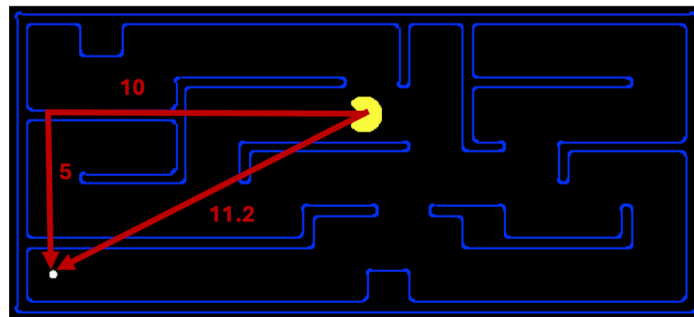
1. 搜索启发式 Search Heuristics

a. 启发式 Heuristics: 一个函数，用于估计当前状态到目标状态的距离 how close a state is to a goal。设计用于特定的搜索问题，不同问题需要不同的启发式。

i. 曼哈顿距离 Manhattan distance: 适用于网格地图上的最短路径问题。

ii. 欧几里得距离 Euclidean distance: 适用于计算两点间的直线距离。

b. 案例

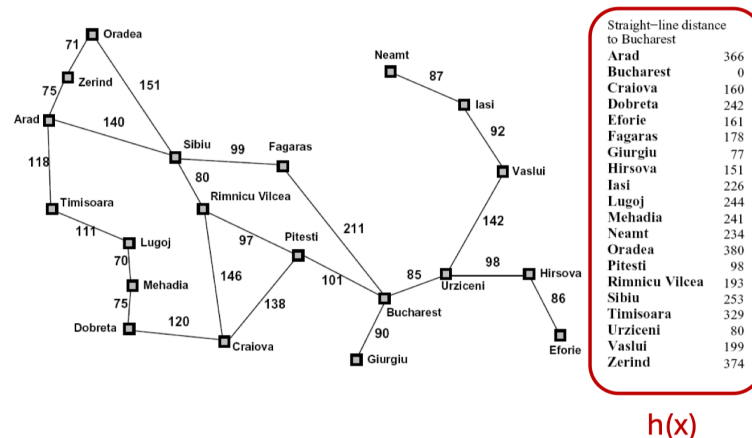


i. Manhattan distance: 沿网格水平方向和垂直方向计算，总长为 $10+5=15$ 。

ii. Euclidean distance: 直接计算两点之间的直线距离，为 11.2。

2. 启发式函数 Heuristic Function $h(x)$

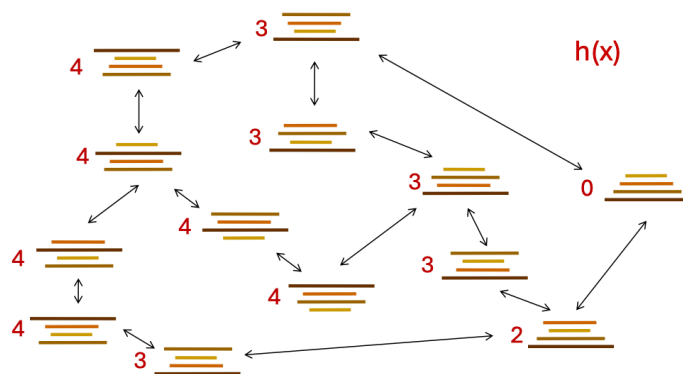
a. 案例



左侧的图展示了罗马尼亚的城市网络，每个节点代表一个城市，边上的数字表示两个城市之间的实际距离。目标城市是 Bucharest，其他城市与其相连，并标注了路径距离。右侧表格显示了各城市到布加勒斯特的直线距离

Straight-line distance to Bucharest. 这个启发式函数 $h(x)$ 用于估计当前城市到目标城市 Bucharest 的代价。

b. 案例



煎饼排序问题涉及将一堆大小不同的煎饼按大小顺序从小到大排列。 $h(x)$: 当前状态下, 仍然未放置到正确位置的最大煎饼的编号。如果最大的煎饼已经放置正确, 则 $h(x) = 0$ 。如果最大的煎饼仍然不在正确的位置, 则 $h(x)$ = 该煎饼的编号。

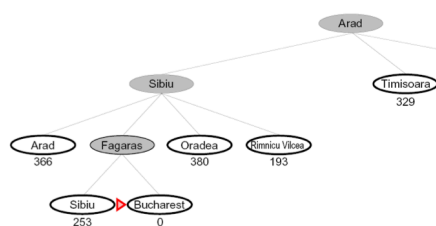
Greedy Search 和 A* Search

3. Greedy Search 的策略与实现

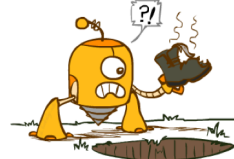
- Strategy:** 在搜索过程中, 每次都选择当前看起来最接近目标的节点进行扩展。通常使用 Heuristic Function $h(x)$ 来评估当前节点到目标的直线距离。
- Common case:** Greedy Search 可能会直接选择错误的目标路径, 因为它仅依赖 Heuristic Function, 而不考虑真实的路径代价。Greedy Search 可能像一个糟糕的 DFS。只看 Heuristic Function, 而不考虑整体路径代价, 可能导致搜索陷入一个深度方向, 最终找到的解不是最优解。

Greedy Search

- Expand the node that seems closest...



- What can go wrong?



4. A* Search 的策略与实现

a. Strategy

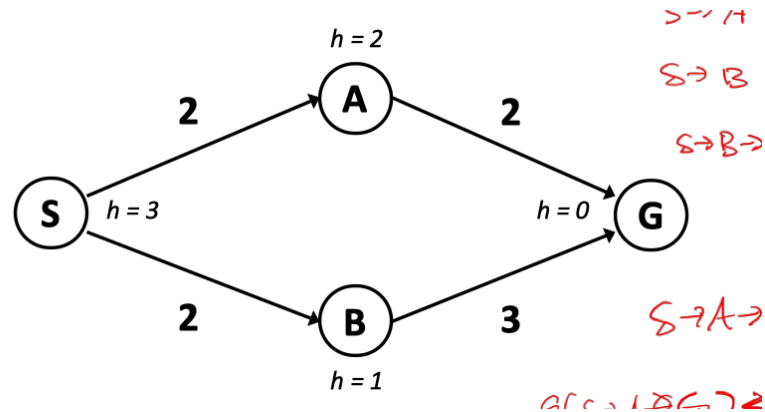
- 我们使用 UCS 和 Greedy search 一起构建 A* search。UCS 是依据路径代价进行排序，选择当前路径代价最小的 node 扩展。Greedy search 是依据 $h(n)$ （估计的目标距离）进行排序，选择当前看起来最接近目标的 node 进行扩展。
- A* search 结合路径代价 $g(n)$ 和 Heuristic Function $h(x)$

$$f(n) = g(n) + h(n)$$

A* search 选择 $f(n)$ 最小的 node 进行扩展。

b. When should A* terminate?

- A* search 不应该在 goal node 被加入队列时终止，而应该等到它被从队列中取出 dequeue 时才终止。当 goal node 第一次被加入队列时，并不一定是最优路径！可能稍后会找到更短的路径到达 goal node，因此不应立即终止。
- 案例



- Initial State

$$f(A) = g(A) + h(A) = 2 + 2 = 4$$

$$f(B) = g(B) + h(B) = 2 + 1 = 3$$

选择扩展 B，因为 $f(B)=3$ 最小

- 扩展 B

$$f(G) = g(G) + h(G) = 5 + 0 = 5$$

目标节点 G 被加入队列，但不终止，因为可能有更短路径。

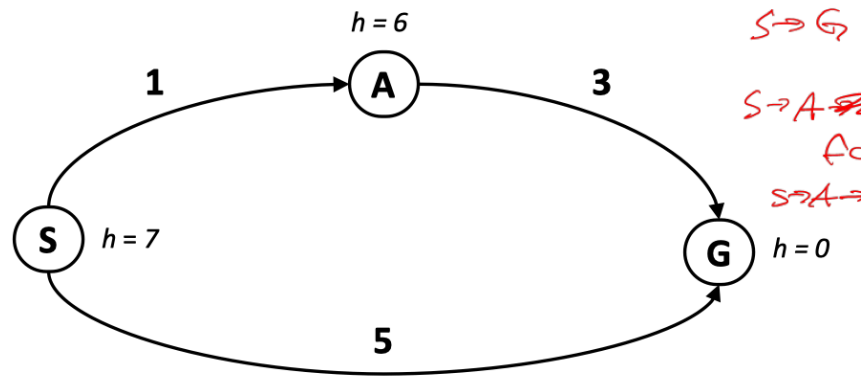
- 扩展 A

$$f(G) = g(G) + h(G) = 4 + 0 = 4$$

这比之前通过 B 到 G 的路径更短，因此此时我们真正的扩展节点 G，搜索结束。最终机器选择了从 A 扩展到 G。

c. Is A* Optimal?

- i. 当 Heuristic Function $h(x)$ 大于实际路径代价 $h(n) > g(n)$ 时, A^* 可能无法找到最优解。
- ii. 案例



- Initial State

从 A 到 G $f(A) = g(A) + h(A) = 1 + 6 = 7$

直接到 G $f(G) = g(G) + h(G) = 5 + 0 = 5$

因此我们优先扩展 G 而不是 A。

- 由于 A^* 选择 f 最小的 node 扩展, 它会扩展 G 而不是 A, 导致了次优路径 S—G ($f(n)=5$), 而不是更短的 S—A—G ($f(n)=4$)。
- 这里 A^* 失败的原因是 启发式 h 估计过大, 导致 A^* 优先扩展错误的路径。

5. Admissible Heuristics

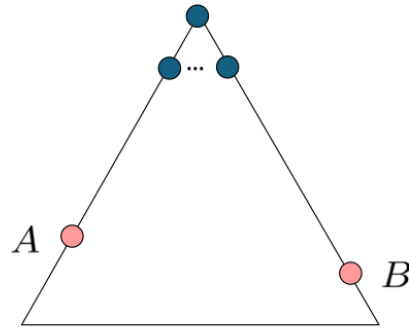
- a. **Admissibility:** 我们可以衡量 Heuristics 的 Admissibility, inadmissible heuristics 低估了最优解的价值, 导致搜索算法卡在次优路径, 无法找到真正的最优解; admissible heuristics 永远不会超过实际的最优代价, 可能会稍微拖慢最优路径的探索 (因为有时会考虑较远的路径), 但它保证最终找到全局最优解。
- b. **Definition of admissible heuristics:** 一个 heuristics $h(n)$ 是 admissible, 如果它对所有 node n 都满足

$$0 \leq h(n) \leq h^*(n)$$

$h(n)$ 是启发式估计的成本; $h^*(n)$ 是从节点 n 到最近目标的真实成本, 是当前节点 n 到目标节点 G 之间的最短实际代价 (最优路径; 也就是最小的 $g(n)$ 值)。

- c. A^* 搜索的成功 取决于启发式的设计。可采纳启发式确保找到最优解, 但如果估计过低, 可能会让搜索变得更慢。如果启发式高估了实际代价 (即 不可采纳), A^* 可能会错过最优解。

6. Optimality of A* Tree Search 的证明



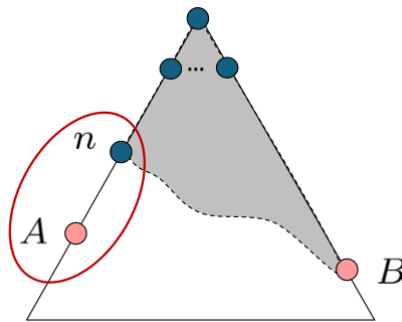
假设 A 是 optimal goal node, B 是 suboptimal goal node, h 是 admissible 的。我们要证明在 h 是 admissible 的情况下, A* 一定可以找到最优解。

- a. 我们知道 B 作为 suboptimal goal node, 肯定在 fringe 中。某个 A 的 ancestor n 也在 fringe。首先, 根据 Admissibility 的定义, 我们可知 $0 \leq h(n) \leq h^*(n)$ 。在 A* 中, $f(n) = g(n) + h(n)$, 由于 $h(n) \leq h^*(n)$, 因此

$$g(n) + h(n) \leq g(n) + h^*(n)$$

由于 $h^*(n)$ 是 n 到目标 A 的实际最短路径代价, 所以 $g(n) + h^*(n) = g(A)$ 。因此, $g(n) + h(n) = f(n) \leq g(A)$ 。因为在 goal node 处, $h=0$, 因此

$$f(n) \leq f(A)$$



- b. 由于 b 是 suboptimal, 因此 $g(A) < g(B)$, 而 $f(A) < f(B)$ 。
c. 因此我们可得, $f(n) \leq f(A) < f(B)$ 。这就说明, n 会在 B 之前被扩展。由于 all ancestors of A expand before B, 这 A expands before B。这说明 A* 是最优解。

7. Consistent Heuristics

- a. Definition of consistent heuristics: Consistent heuristics 必须满足三角不等式, 即一个 node x 到其邻居 y 的 heuristics 差值不能超过实际代价 $g(x, y)$ 。

$$h(x) - h(y) \leq g(x, y)$$

- b. 作用: Consistent 确保沿最优路径“稳步前进”, 避免因代价不一致而反复访问同一节点。A* 不需要重新访问已扩展的节点, 从而减少了扩展次数和内存消耗。在一致性条件下, A* 仍然保证最优解的找到。

8. Consistency implies admissibility

设 n 到目标的最优路径为

$$n = n_0 \rightarrow n_1 \rightarrow n_2 \rightarrow \dots \rightarrow n_k = \text{Goal}$$

其中每条边 $n_i \rightarrow n_{i+1}$ 的 cost 为 $g(n_i, n_{i+1})$ 。根据 Consistency

$$h(n_i) \leq g(n_i, n_{i+1}) + h(n_{i+1})$$

对路径 $n = n_0 \rightarrow n_1 \rightarrow n_2 \rightarrow \dots \rightarrow n_k$ 逐步相加

$$h(n) \leq \sum_{i=0}^{k-1} g(n_i, n_{i+1}) + h(\text{Goal})$$

由于 $h(\text{Goal}) = 0$

$$h(n) \leq \sum_{i=0}^{k-1} g(n_i, n_{i+1}) = h^*(n)$$

9. Properties of A* Search

a. UCS vs A* Contours

- i. Uniform-cost expands equally in all “directions”. UCS 扩展的方式是等距离向四周扩展，不偏向目标方向。
- ii. A* expands mainly toward the goal, but does hedge its bets to ensure optimality. 因此，A* 更倾向于朝向目标扩展，但也会适当向周围扩展，以确保搜索的最优性（如右侧的扁平扩展区域）。

b. Summary

- i. A* uses both backward costs and (estimates of) forward costs.
- ii. A* is optimal with admissible / consistent heuristics.
- iii. Heuristic design is key: often use relaxed problems.

Lecture 4: Probability

Random Variables and Distributions

1. Random Variables

- a. Definition: Random Variables 是用来表示我们对世界某些不确定性方面的变量。
 - i. $R = \text{Is it raining?}$
 - ii. $T = \text{Is it hot or cold?}$
 - iii. $D = \text{How long will it take to drive to work?}$
 - iv. $L = \text{Where is the ghost?}$
- b. 表示方法: Random variables 通常用大写字母表示, 比如 R 、 T 、 D 、 L 。
- c. Domains: 每个随机变量都有其取值范围
 - i. $R \in \{true, false\}$
 - ii. $T \in \{hot, cold\}$
 - iii. $D \in [0, \infty]$

2. Probability Distributions

- a. Probability Distributions 表示每个随机变量的不同取值发生的可能性。概率的和必须是 1。
- b. Distribution: A distribution is a TABLE of probabilities of values。
- c. Probability: A probability (lower case value) is a single number。

$P(T)$		$P(W)$	
T	P	W	P
hot	0.5	sun	0.6
cold	0.5	rain	0.1
		fog	0.3
		meteor	0.0

- d. 案例一: Temperature
 - i. $P(T = hot) = 0.5$
 - ii. $P(T = cold) = 0.5$
 - iii. 图示展示了机器人在热和冷两种温度下的反应。
- e. 案例二: Weather
 - i. $P(W = hot) = 0.6$
 - ii. $P(W = rain) = 0.1$
 - iii. $P(W = fog) = 0.3$
 - iv. $P(W = meteor) = 0.0$
 - v. 图示展示了不同天气下机器人的表现。
- f. 有效性条件

- i. 每个概率必须是非负 $P(X = x) \geq 0$
- ii. 所有可能性大总和必须等于 1 $\sum_x P(X = x) = 1$

3. Joint Distributions

- a. **Definition:** 一个 Joint Distribution 定义了多个 random variable 同时取某个特定取值时的概率。例如，温度 T 和天气 W 组成的联合分布 $P(T, W)$ 表示 T 和 W 同时取某个特定取值时的概率。

$$P(T, W)$$

T	W	P
hot	sun	0.4
hot	rain	0.1
cold	sun	0.2
cold	rain	0.3

- b. **Size of distribution:** 如果有 n 个 variables，每个变量的 domain 是 d，则 joint distribution 的大小为 d^n 。例如对于 T 和 W 两个二值变量，表格大小为 4。

4. Probabilistic Models 与 Constraint Satisfaction Problems

Distribution over T,W

T	W	P
hot	sun	0.4
hot	rain	0.1
cold	sun	0.2
cold	rain	0.3

Constraint over T,W

T	W	P
hot	sun	T
hot	rain	F
cold	sun	F
cold	rain	T

- a. A probabilistic model is a joint distribution over a set of random variables.
 - i. Random Variables 具有不同的取值范围 domains。
 - ii. Variable 的某个 assignment 称为 outcome。
 - iii. Joint Distribution: 表示某个赋值发生的可能性大小。
 - iv. Normalized: 所有可能结果的概率和为 1。
 - v. 独立性: 理想情况下，只有某些变量之间存在直接相互作用。
- b. Constraint satisfaction problems: 通过约束条件 Constraint 定义哪些变量赋值是可行的，不涉及概率。
 - i. 变量有固定取值范围。
 - ii. 约束 Constraint: 定义哪些变量组合是允许的，哪些是禁止的。
 - iii. 不要求概率总和为 1，只需要检查解的可行性。

5. Events 与 Sample Space

- a. Events 是一组可能的 outcomes，即多个（但不一定是全部）assignment 的集合。事件的概率 $P(E)$ 是所有满足事件条件的赋值概率之和。比如

$$P(T, W)$$

T	W	P
hot	sun	0.4
hot	rain	0.1
cold	sun	0.2
cold	rain	0.3

- i. 事件 hot AND sunny

$$P(T = \text{hot}, W = \text{sun}) = 0.4$$

- ii. 事件 hot

$$P(T = \text{hot}) = P(\text{hot}, \text{sun}) + P(\text{hot}, \text{rain}) = 0.4 + 0.1 = 0.5$$

- b. 样本空间 Sample Space

- i. 所有可能的赋值组合构成的全集，概率之和必然等于 1。例如，对于

$T \in \{\text{hot}, \text{cold}\}, W \in \{\text{sun}, \text{rain}\}$ 而言，样本空间包含：

$$\Omega = \{(\text{hot}, \text{sun}), (\text{hot}, \text{rain}), (\text{cold}, \text{sun}), (\text{cold}, \text{rain})\}$$

- ii. Events 是 Sample Space 的一个子集，表示我们感兴趣的某些情况。

- c. Example

$$P(X, Y)$$

X	Y	P
+x	+y	0.2
+x	-y	0.3
-x	+y	0.4
-x	-y	0.1

- i. $P(+x, +y) = 0.2$

- ii. $P(+x) = P(+x, +y) + P(+x, -y) = 0.2 + 0.3 = 0.5$

- iii. $P(-y \text{ OR } +x) = P(-y) + P(+x) - P(-y, +x) = 0.4 + 0.5 - 0.3 = 0.6$

6. Marginal Distributions

- a. Definition: Marginal distributions are sub-tables which eliminate variables，通过消除（summed out）某些变量来得到其他变量的概率分布。

- b. Method: 通过对联合分布中不感兴趣的变量的概率求和来实现。

$P(T, W)$			$P(T)$	
T	W	P	T	P
hot	sun	0.4	hot	0.5
hot	rain	0.1	cold	0.5
cold	sun	0.2	$P(W)$	
cold	rain	0.3	W	P
			sun	0.6
			rain	0.4

c. 边际化 Marginalization 的一般公式

$$P(X_1 = x_1) = \sum_{x_2} P(X_1 = x_1, X_2 = x_2)$$

7. Conditional Probabilities

a. Definition: 给定 Event B 发生的前提下, Event A 发生的概率, 表示为

$$P(a|b) = \frac{P(a, b)}{P(b)} = \frac{\text{Joint Distributions of } a \text{ and } b}{P(b)}$$

当我们知道 b 发生后, 重新考虑 a 发生的概率。好比在得知下雨的情况下, 天气变冷的概率是多少。

b. Example:

$P(T, W)$		
T	W	P
hot	sun	0.4
hot	rain	0.1
cold	sun	0.2
cold	rain	0.3

$$P(W = \text{sun} | T = \text{cold}) = \frac{P(W = \text{sun}, T = \text{cold})}{P(T = \text{cold})} = \frac{0.2}{0.2 + 0.3} = 0.4$$

8. Conditional Distributions

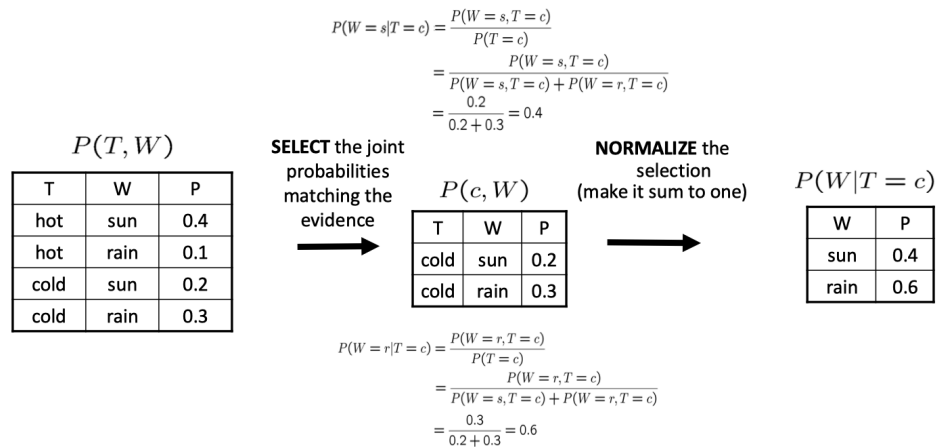
a. Definition: 给定某个变量的取值, 计算另一个变量的概率分布。它是通过联合分布 $P(T, W)$ 归一化得到的。

$$P(x_1 | x_2) = \frac{P(x_1, x_2)}{P(x_2)} = \frac{P(x_1, x_2)}{\sum_{x_1} P(x_1, x_2)}$$

$P(x_1, x_2)$ 是联合概率; $P(x_2)$ 是边际概率, 也就是通过对所有 x_1 可能取值下的联合概率求和得到的。

$$P(x_2) = \sum_{x_1} P(x_1, x_2)$$

- b. **Normalization Trick:** 如何通过联合分布计算条件分布，即在某些条件下（比如 $T=cold$ ）的分布，我们可以使用归一化技巧 Normalization Trick
- i. 比如我们要计算 $P(W|T = c)$ 。



9. Normalization

- a. **Dictionary Definition:** 将所有数值调整到正常的条件 (normal condition)。
- b. **数学定义:** 将所有条目除以它们的总和，使得它们的和为 1，从而形成概率分布。
- i. Step 1: Compute $Z = \text{sum over all entries}$
- ii. Step 2: Divide every entry by Z

• Example 1

W	P
sun	0.2
rain	0.3

Normalize
 $Z = 0.5$

W	P
sun	0.4
rain	0.6

• Example 2

T	W	P
hot	sun	20
hot	rain	5
cold	sun	10
cold	rain	15

Normalize
 $Z = 50$

T	W	P
hot	sun	0.4
hot	rain	0.1
cold	sun	0.2
cold	rain	0.3

10. **Probabilistic Inference:** 通过已知的概率，推导出某个事件的概率。

11. The Product Rule

$$P(y)P(x|y) = P(x, y)$$

12. The Chain Rule

$$P(x_1, x_2, x_3) = P(x_1)P(x_2|x_1)P(x_3|x_1, x_2)$$

13. Bayes' Rule

$$P(x|y) = \frac{P(y|x)}{P(y)}P(x)$$

14. 全概率公式 Law of Total Probability

$$P(A) = \sum_{i=1}^n P(A|B_i) \times P(B_i)$$

Lecture 5: Constraints Satisfaction Problems

搜索的意义与约束满足问题 The Meaning of Search and Constraint Satisfaction Problems

1. What is search for?

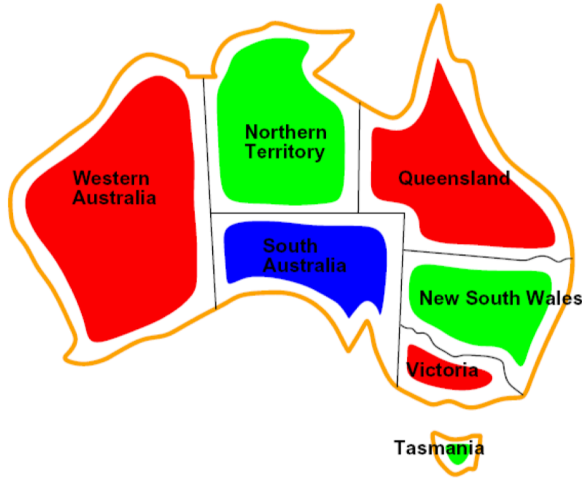
- a. Assumptions about the world: a single agent, deterministic actions, fully observed state, discrete state space.
- b. 第一类用途: Planning 规划问题
 - i. 目标是找到 the path 来达成目标 The path to the goal is the important thing。
 - ii. Paths have various costs, depths。
 - iii. 可使用 heuristics 来提高搜索效率。
 - iv. 比如有一个机器人: 计划路径去偷宝石——强调路径 Path 本身。
- c. 第二类用途: Identification 识别问题
 - i. 只关注最终的变量赋值, 不关心达到这个赋值的路径。The goal itself is important, not the path。
 - ii. 所有解的路径长度可能相同。
 - iii. 约束满足问题 CSPs are specialized for the identification problems。
 - iv. 比如有一个机器人: 在寻找宝石的位置——强调结果, 而非过程。

2. Constraint Satisfaction Problems

- a. Standard search problems
 - i. State is black box: state 可以是任意复杂的数据结构 (如路径、棋局等)。
 - ii. Goal test: 可以是对 state 的任意函数判断。
 - iii. Successor function: 定义可以非常自由, 完全自定义下一步如何生成。
 - iv. Standard search problem 的灵活性很高, 但也意味着我们无法利用问题结构来进行通用优化。
- b. Constraint satisfaction problems (CSPs)
 - i. A special **subset** of search problems。
 - ii. 每个 state 由一组 variables X_i 构成, variable 的取值来自一个 domain D (不同 variable 也可以有不同的 domain)。
 - iii. Goal test 由一组约束 constraints 组成, 规定了某些 variable 的取值组合是否合法。
 - iv. CSP 的优势
 - CSP 是一种 formal representation language, 其清晰、简洁、结构化。

- Allows useful general-purpose algorithms with more power than standard search algorithms。

c. CSP 案例 1



- Variable: 每个 state 由一组 variable 构成，在这里 variable 是
 - WA (Western Australia)
 - NT (Northern Territory)
 - Q (Queensland)
 - NSW (New South Wales)
 - V (Victoria)
 - SA (South Australia)
 - T (Tasmania)
- Domain: Variable 的取值来自一个 domain D，在这里每个 variable (即区域) 都可以取值
 - Red
 - Green
 - Blue
- Constraint: Goal test 由一组约束 constraints 组成，在这里核心约束条件是相邻区域不能有相同颜色
 - Implicit constraint: 使用数学表达进行书写，比如

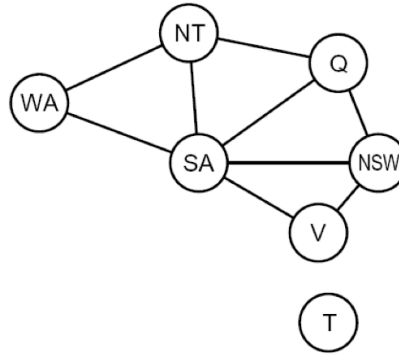
$$WA \neq NT \dots$$
 - Explicit constraint: 把允许的配对列出来，比如

$$(WA, NT) \in \{(red, green), (red, blue), (green, red), \dots\}$$
- Solution: 一个 solution 是所有 variable 的赋值，满足所有 constraint 的组合，比如

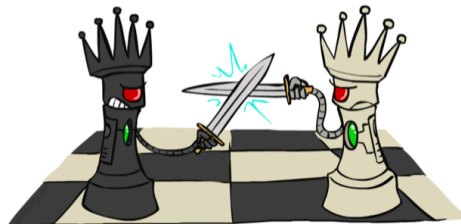
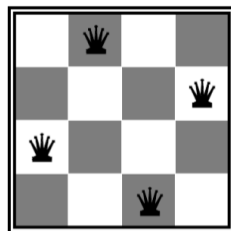
$$\{WA=red, NT=green, Q=red, NSW=green, V=red, SA=blue, T=green\}$$

d. 二元约束与二元约束图 Binary CSP and Binary Constraint Graph

- i. Binary CSP: 指每个 constraints 只涉及两个 variables。比如之前的案例中，约束条件为相邻区域不能有相同颜色，我们只涉及两个 variables（两个相邻区域不能有相同颜色）。
- ii. Binary constraint graph



- Nodes: 代表 variables，例如 WA, NT, Q, NSW 等。
 - Arcs: 代表两个 variables 之间存在 constraints（例如相邻区域不能有相同颜色）。
 - 独立子问题 Independent subproblem: 在图中，T 是一个孤立的点 —— 表示它与其他区域没有 constraints（不相邻），所以可以独立求解。
- e. 一个问题可以有多个 CSP formulation，以下面这个问题为例：目标是在一个 $N \times N$ 的棋盘上放置 N 个皇后，使得任意两个皇后不在同一行、不在同一列、不在同一个对角线。



i. N-Queens 问题的 CSP 表述方式之一（Formulation 1）

- Variables: 用一个 $N \times N$ 的 matrix of boolean variables X_{ij} 表示棋盘。比如 X_{11} 代表第一行第一列。
- Domains: 每个 variable $X_{ij} \in \{0,1\}$ ，即每格要么放皇后，要么不放。
- Constraints

- a. 每一行最多只能有一个皇后：对于同一行中任意两个位置 $(i, j), (i, k)$ ，必须满足如下条件（不能出现 $(1,1)$ ）。

$$(X_{ij}, X_{ik}) \in \{(0,0), (0,1), (1,0)\}$$

- b. 每一列最多只能有一个皇后：对于同一列任意两个位置 $(i, j), (k, j)$ ，必须满足如下条件（不能出现 $(1,1)$ ）。

$$(X_{ij}, X_{kj}) \in \{(0,0), (0,1), (1,0)\}$$

- c. 主对角线（从左上到右下的方向）

$$(X_{ij}, X_{i+k, j+k}) \in \{(0,0), (0,1), (1,0)\}$$

- d. 副对角线（沿着从右上到左下）

$$(X_{ij}, X_{i+k, j-k}) \in \{(0,0), (0,1), (1,0)\}$$

- e. 数量约束（只能放 N 个皇后，表示总共有 N 个位置被标为 1）

$$\sum_{i,j} X_{ij} = N$$

ii. N-Queens 问题的 CSP 表述方式之一（Formulation 2）

- Variables: 每个 variables Q_k 表示在 k 行第几个皇后的位置，例如 Q_1 表示第一行的皇后放在哪一列。
- Domain: $Q_k \in \{1, 2, \dots, N\}$ ，即皇后可以放在该行的第 1 到第 N 列中的某一个位置。
- Constraints

- a. Implicit: 用数学条件定义

$$Q_i \neq Q_j \text{ (两个皇后不能在同一列)}$$

$$|Q_i - Q_j| \neq |i - j| \text{ (不在同一对角线)}$$

- b. Explicit: 可以用值对的方式列出所有允许的组合，比如：

$$(Q_1, Q_2) \in \{(1,3), (1,4), \dots\}$$

f. Constraints 的两种表达方式

- i. 基础约束 Basic constraints: 通常是 pairwise inequality constraints（成对不等式），这意味着每个 constraint 只涉及两个 variables。

- 比如 N-Queens 问题的 $Q_i \neq Q_j$ 或者 $|Q_i - Q_j| \neq |i - j|$ ，他们都只涉及两个 variables，属于 basic constraints。
- 再比如 N-Queens 问题的 $(X_{ij}, X_{ik}) \in \{(0,0), (0,1), (1,0)\}$ ，这也只涉及两个 variables，属于 basic constraint。

- ii. 高级约束 High-level constraints: 通常设计多个 variables，更加抽象，表达能力更强。通常 CSP 内部会将其转化为多个 basic constraints。

- alldiff (所有 variables 取值各不相同): 比如对于数独, 每行使用一个 alldiff($X_1, X_2, \dots, X_9$), 如果想要转化为 basic constraints, 则写为 $X_1 \neq X_2, X_1 \neq X_3, \dots$ 。
- sum (一组变量加起来等于某个值): 比如对于 N-Queens 问题的 $\sum_{i,j} X_{ij} = N$, 其就是一个 high-level constraints, 也可以被写为 $\text{sum}([X_{11}, X_{12}, \dots, X_{NN}]) = N$ 。

g. CSP 的多种变体 Varieties of CSPs

i. Discrete Variables

- Finite domains: 每个 variables 的值来自一个 finite 集合, 例如

$$D = \{red, green, blue\}$$
 搜索空间大小为 $O(d^n)$, 其中 d 是 domain 的大小, n 是 variable 个数。
- Infinite domains: Variables 可以取无限多个数字, 比如整数范围内、实数范围内等等。比如 job scheduling 问题中, 假设我们有三个任务 Task A、B 和 C, 我们的 variables 可以是 S_A, S_B, S_C 。这三个变量可以是任务 A、B 和 C 的开始时间 (也可以使用 E_A, E_B, E_C 来表示结束时间)。这个时候, 我们每个 variables 的数值来自一个 infinite domains, 例如任意非负整数。

ii. Continuous Variables

- 通常用于物理学或者工程学领域。
- Variables 可以取实数, 例如 start/end times for Hubble Telescope observations。

iii. 硬约束问题 Hard Constraints

- 一元约束 Unary Constraints: 只涉及一个 variable, 相当于直接对 variable 的 domain 做出限制, 比如 $SA \neq green$ 。
- 二元约束 Binary Constraints: 涉及两个 variables, 是 CSP 中最常见的类型, 例如在地图着色问题中 $SA \neq WA$ 。
- 三元或更多约束 Higher-Order Constraints: 涉及三个或更多变量, 例如数独中的 alldiff($X_1, \dots, X_9$)。

iv. 软约束问题 Soft Constraints/Preferences

- 不要求“必须满足”, 而是“希望满足得更好”。
- 比如, red 比 green 更好。这个时候, 我们可以说 red 的 preference 更大, 其 cost 更低, green 的 preference 更小, 其 cost 更高。因此, 其可表示为每个 variables 赋值的 cost。
- 这类问题也被称为约束优化问题 Constrained Optimization Problems。

h. 真实世界中的 CSPs

- i. Assignment Problems: 谁去教哪一门课? Variables 可以是老师, domains 就是课程。
- ii. Timetabling Problems: 哪门课排在哪一天、哪个时间?
- iii. Hardware Configuration: 如何组合兼容的组件来构建一台系统(例如笔记本电脑定制), 约束可能是显卡必须匹配主板插槽、电源功率不能过载。
- iv. Transportation Scheduling: 公交或火车的班次安排、司机轮班、车辆调度等。
- v. Factory Scheduling: 哪个产品在什么时候在哪台机器上加工?
- vi. 许多现实问题的 variable 都不是 discrete 的, 而是 continuous 的, 因此这涉及到 continuous CSP。用 LP(线性规划)或 CP(约束编程)工具来求解。

3. 回溯搜索 Backtracking Search

a. 什么是 Backtracking Search

- i. 基于 DFS 的搜索算法, 专门为 CSP 设计“在搜索过程中逐步赋值并检查 constraints, 一旦有冲突立刻回溯”。
- ii. One Variable at a Time: 一步只给一个 variable 赋值, 赋值顺序是无关的(commutative), 所以我们可以固定顺序来简化搜索空间。
- iii. Check Constraints as You Go: 每赋一次值, 就立刻检查是否与之前的赋值冲突。如果冲突, 即可剪枝(不在尝试拓展该分支)。这个过程叫 incremental goal test。

b. Backtracking Search 案例: 假设现在有 A、B 和 C 三个区域, A 与 B 相邻, B 与 C 相邻(A 与 C 不相邻), 每个区域可以选择 $Domain = \{Red, Green\}$ 。

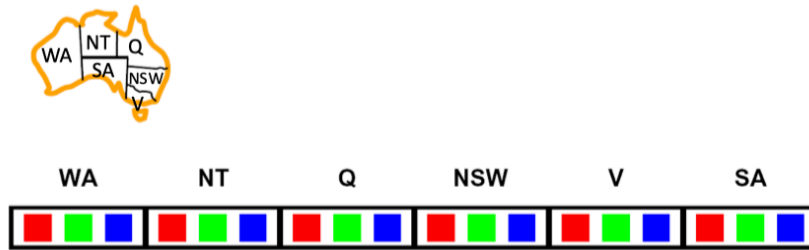
- i. Initial state: {} 空赋值
- ii. Step 1: 给 A 赋值 {A: Red}, 目前只有一个 variable, 无冲突。
- iii. Step 2: 给 B 赋值
 - B=Red, 冲突, 回溯
 - B=Green, 不冲突, assignment = {A: Red, B: Green}。
- iv. Step 3: 给 C 赋值。
 - C=Red, 不冲突, assignment = {A: Red, B: Green, C: Red}。
- v. 最终解: {A: Red, B: Green, C: Red}。

c. Improving Backtracking

- i. 顺序优化 Ordering: 决定下一个要赋值的 variable 是谁或者决定尝试哪些值的顺序。
- ii. 前向检测 Filtering: 在赋值后, 立刻检查是否会导致将来失败。

iii. 结构利用 Structure: 是否可以拆分成若干子问题?

- d. Forward Checking: 当你给一个 variable 赋值之后, 立刻检查它对尚未赋值的其他 variable 有没有影响。从这些未赋值 variable 的 domain 中, 删除那些与当前赋值冲突的值。



- 一旦我们给某个区域赋了颜色, 比如 $WA = \text{red}$, 就立刻把和 WA 相邻的 NT 和 SA 的 red 的值删掉。
- 如果某个未赋值变量 被删到没有可选值了, 说明当前路径必然失败, 直接回溯!

e. Constraint Propagation

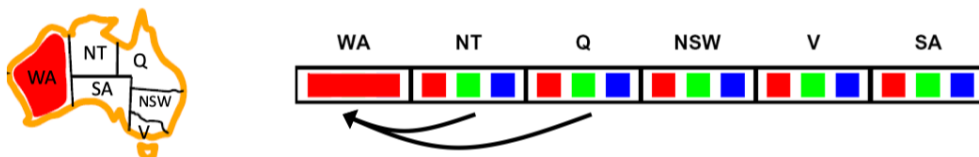


- Forward checking 只在赋值后的直接邻居中删值, 但很多冲突是间接传导的, forward checking 看不到, 所以你会错过某些“注定失败”的分支。
- Constraint Propagation 是一种更强的推理机制, 从一个约束推到另一个, 再推到另一个。比如如果 $WA = \text{red} \rightarrow NT \neq \text{red} \rightarrow NT = \text{blue} \rightarrow SA \neq \text{blue} \rightarrow SA$ 无合法值 $\rightarrow X$ 。

4. Arc Consistency and AC-3

- Arc Consistency: 一个 arc $X \rightarrow Y$ 是 consistent 当且仅当, 对于 X domain 中的每一个 x , 都至少存在一个 Y 的值 y , 使得 x, y 组合不违反 constraints。
 - 也就是说, X 的所有选项都“有路可走”。
 - 如果某个 $x \in D_X$ 没有任何 $y \in D_Y$ 使得 constraints 成立, 就立刻删除这个 x 。

b. 案例 Example



比如我们现在关注 $WA \rightarrow NT$ ，假设 WA 的 domain 是 $\{\text{Red}\}$ ，我们检查 $WA \rightarrow NT$ ，发现 NT 的 domain 是 $\{\text{Red, Green, Blue}\}$ ，而 constraint 告诉我们 WA 不能等于 NT ，也就说明 NT 不能是 Red ，但是 NT 还有 Green 和 Blue ，说明 $WA \rightarrow NT$ 是 consistent。

c. AC-3 算法

- i. A simple form of propagation makes sure all arcs are consistent。我们不只检查新赋值相关的邻接 arc，这是不断传播，直到每一条 arc 都满足 arc consistency。
- ii. 在我们解 CSP 过程中，我们使用的 AC-3 算法如下。案例为，给定一个地图 $A - B - C$ ，constraint 为 $A \neq B$ ， $B \neq C$ 。初始 domain 为 $D(A) = D(B) = D(C) = \{\text{red, green}\}$ 。

- Step 1: 写出队列 Q 的备选 arc $[A \rightarrow B, B \rightarrow A, B \rightarrow C, C \rightarrow B]$ 。
- Step 2: 选取第一个 variable，随机赋值。比如我们选择 $D(A) = \{\text{red}\}$ 。现在我们需要将包含 A 的 arc 放入队列中。

$$Q = [A \rightarrow B, B \rightarrow A]$$

我们需要先处理指向第一个 variable 的 arc，即 $B \rightarrow A$ 。我们检查 B 中的值是否能找到 A 中的不冲突值。 $D(B) = \{\text{red, green}\}$ ，但 $A = \{\text{red}\}$ ，因此我们从 B 中删除 red 。

$$D(B) \leftarrow \{\text{green}\}$$

现在 B 的 domain 变了，我们需要把影响 B 的弧重新放入队列 Q 。因此，现在

$$Q = [A \rightarrow B, B \rightarrow A, B \rightarrow C, C \rightarrow B]$$

- Step 3: 继续处理剩下的 $A \rightarrow B$ ，我们发现 $A = \{\text{red}\}$ 、 $B = \{\text{green}\}$ ，因此满足 constraint，无需剔除。
- Step 4: 处理指向第二个 variable 的 arc，即 $C \rightarrow B$ 。我们要确保 C 中每个值都可以与 B 的值配对。 $C = \{\text{red, green}\}$ 中的 green 冲突了，因此被剔除。

$$D(C) \leftarrow \{\text{red}\}$$

- Step 5: 继续处理剩下的 $B \rightarrow C$ ，我们发现 $B = \{\text{green}\}$ 、 $C = \{\text{red}\}$ ，因此满足 constraint，无需剔除。
- Step 6: 最终结果为 $A = \{\text{red}\}$ 、 $B = \{\text{green}\}$ 、 $C = \{\text{red}\}$ 。

- iii. 在上述过程中，delete from the tail! 是核心原则，因此我们要先处理 $B \rightarrow A$ 再处理 $A \rightarrow B$ ，先处理 $C \rightarrow B$ ，再处理 $B \rightarrow C$ 。

- iv. Time complexity: $O(n^2 d^3)$ ，可优化至 $O(n^2 d^2)$ 。

- v. Limitations of Arc Consistency

- 我们无法保证问题有解，也不能直接给出所有解，有时甚至明明无解但我们还不知道。
- Arc consistency still runs inside a backtracking search!

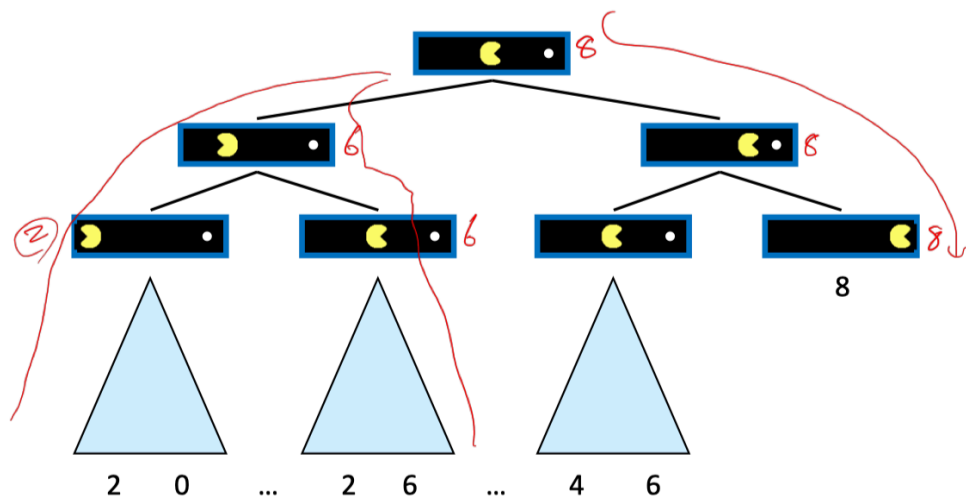
Lecture 6/7: Game Theory and Utilities

Types of Games

1. Many different kinds of games!
 - a. Deterministic or stochastic: 动作结果是否确定? (如国际象棋和掷骰子)。
 - b. One, two or more players: 玩家数量为单人、对战还是多人博弈。
 - c. Zero-sum: 一方得分是否是另一方损失 (是否是 zero-sum)。
 - d. Perfect information: 游戏状态是否完全可观测 (如国际象棋和扑克牌)。
2. 博弈目标: “策略 (strategy 或 policy)” 是我们想从 algorithms 中计算出来的目标。
 - a. Strategy = 给定任一 state, 推荐采取哪个 action (即 $\text{state} \rightarrow \text{action}$ 的映射)。
 - b. 我们关心的是如何使用算法 (如 Minimax 等) 来计算最优策略。
3. Deterministic Game
 - a. States: S (所有可能的 game state 的集合, 比如国际象棋的棋盘配置)。
 - b. Players: $P = \{1, 2, \dots, N\}$, 玩家通常轮流行动。
 - c. Actions: A (可能执行的动作, 可能依赖于当前 state/当前 player, 比如 Pacman 向左/右/上/下移动)。
 - d. Transition Function: $S \times A \rightarrow S$ 。给定一个 state 和一个 action, 返回下一个 state。因为是 deterministic 的, 所以结果是唯一的。
 - e. Terminal Test: $S \rightarrow \{true, false\}$, 判断某个 state 是否为游戏结束 state。
 - f. Terminal Utilities: $S \times P \rightarrow \mathbb{R}$, 给定一个 terminal state 和一个 player, 返回这个 player 的得分。
4. Zero-Sum Games vs General Games
 - a. Zero-Sum Games
 - i. 玩家之间的收益 utilities 是完全对立的, 一个玩家的收益就是另一个玩家的损失。Utility 总和为 0。
$$\text{player } A's \text{ utilities} = -\text{player } B's \text{ utilities}$$
 - ii. 完全 adversarial, 是 pure competition。
 - iii. 比如两个机器人在抢夺同一个宝石, 谁拿到谁赢, 典型的零和冲突场景。
 - b. General Games
 - i. 玩家各自有独立的效用函数, 不再是严格对立。玩家之间可能合作、互不影响, 或部分竞争。
 - ii. 比如两个机器人在捡不同颜色的宝石, 各自偏好不同颜色, 没有强对抗性, 属于一般博弈。

Single-Agent, Adversarial Game and Minimax

5. Single-Agent Trees



- 叶节点（最底层三角形）Leaf nodes (Terminal States): Utility 是已知的，例如：2, 0, 2, 6, 4, 6, 8。
- 非终止状态（中间和根节点）: 它们的值是通过向上回溯得到的

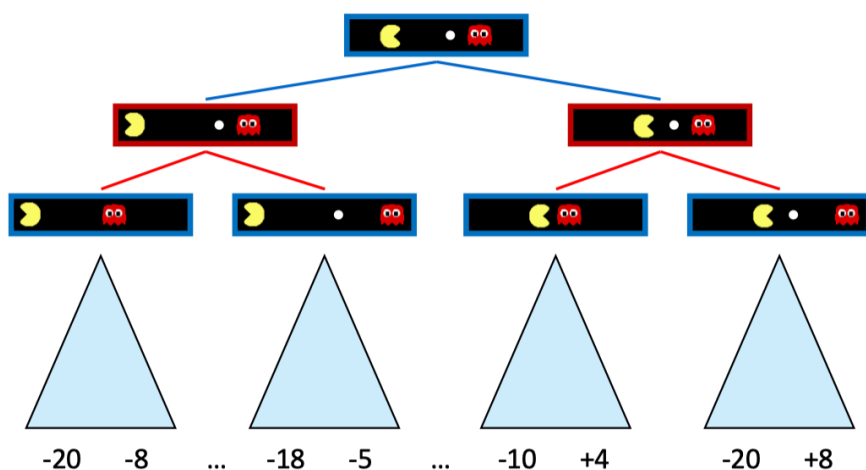
$$V(s) = \max_{s' \in \text{children}(s)} V(s')$$

红色标注说明是如何向上传播的，比如

- 左边的子树，子节点是 2 和 0，父节点取 $\max(2, 0) = 2$ 。
- 右边的子树，子节点是 4 和 6，父节点取 6。
- 整个根节点取其 2 个孩子中的最大值： $\max(6, 8) = 8$ 。

- Terminal States: 其值由游戏规则直接给出 $V(s) = \text{known}$ 。
- Non-Terminal States: 其值需递归计算 $V(s) = \max$ of all children values。

6. Adversarial Game Trees and Minimax



- 蓝色框 (Pac-Man): 是我们的轮次，我们要选择 $\max$; 红色框 (Ghost): 是对手的轮次，他们要选择 $\min$ 。

- i. States under agent's control (蓝色)

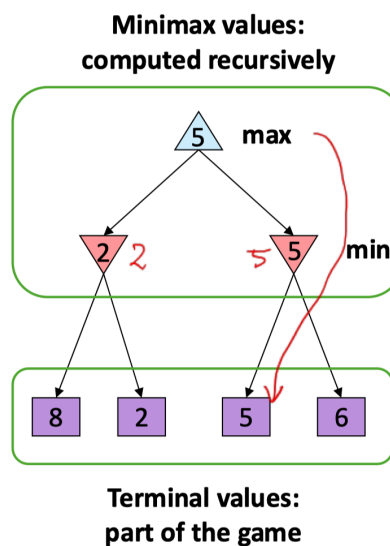
$$V(s) = \max_{s' \in \text{successors}(s)} V(s')$$

- ii. States under opponent's control (红色)

$$V(s') = \min_{s \in \text{successors}(s')} V(s)$$

- b. 数值是如何传播的？

- i. 最下方是蓝色节点，因此选择 max，因此最下方一排蓝色节点的数值为-8、-5、+4 和+8。
- ii. 第二排是红色节点，因此选择 min，因此第二排为-8 和+4。
- iii. 最上面的节点为蓝色，因此选择 max，因此为+4。



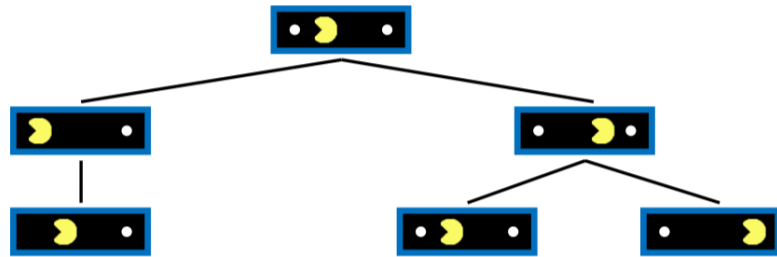
- c. 这就是 Minimax 原理的一个示例，Minimax 是在有理性对手假设下的最优策略。MAX 和 MIN 节点交替，值自底向上传播。
- d. Minimax efficiency and properties
 - i. Minimax 类似于 DFS，其时间复杂度为 $O(b^m)$ ，b 为平均每个 state 的合法 action 数量，m 为 search tree 的最大深度。其空间复杂度为 $O(bm)$ 。
 - ii. 如果对手是 perfect player，Minimax 是最好的策略。因为它总是假设对方会做出最差的决策来最小化你的得分。如果对方不是完美的（比如是随机策略），Minimax 可能太保守。

7. Resource Limits and Depth-Limited Search

- a. 在真实游戏中，无法搜索到树的最底端（叶子结点），比如国际象棋、围棋等。只在树中搜索到有限深度，不再展开全部可能情况。在非终止状态使用 evaluation function 来估值，而不是依赖真实结果（胜/负）。
- b. 但是 evaluation function 是 imperfect 的，可能会导致出错或者产生误导性评分。但是，搜索深度更深可以弥补评估函数的不足。你在搜索树中越往后推

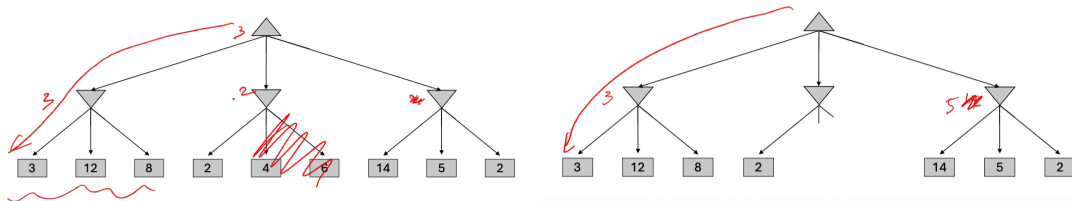
演（层数越深）就意味着你离“真实终局”越近。这里体现了一个 tradeoff，也就是 evaluation function 与 search depth 的权衡。

- i. 精细的 evaluation function 可能需要更多计算（例如用神经网络）。
 - ii. 或者也可以选择更简单的 evaluation function，但加深搜索深度。
- c. Depth-Limited Search 的问题：可能因为短视而陷入局部最优行为（不行动、不得分）。



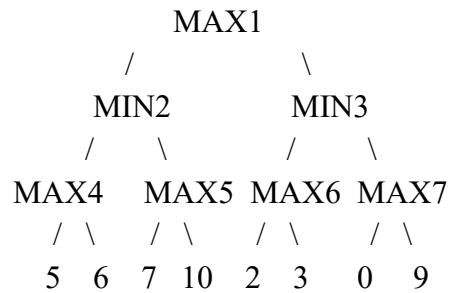
- i. Pacman 知道马上吃掉豆子可以得分。
- ii. Pacman 也知道如果现在不吃，而稍后回来吃，得分一样。
- iii. 在吃掉豆子之后，在当前的搜索深度范围内（例如 depth=2）没有新的得分机会了。
- iv. 所以 Pacman 觉得「现在吃」和「等会吃」是一样好的，于是可能永远不吃！

8. Alpha-Beta Pruning



- a. Alpha-Beta Pruning: 这里有两棵树，但是右边的树通过 pruning 跳过了不必要的搜索，可以节省大量运算。Pruning 不会影响最终结果，只是跳过了不会被选择的子树，显著提升效率。
- b. Alpha-Beta Pruning Properties
 - i. Alpha-Beta Pruning 不会改变最终的 Minimax 值，但中间节点的值可能是不准确的。
 - ii. 子节点排序越好，pruning 效果越强。如果是 perfect ordering（如果我们先访问最优解方向的子节点），那么时间复杂度可以从 $O(b^m)$ 降到 $O(b^{\{m/2\}})$ 。
- c. 如何 Pruning（技巧）
 - i. Pruning 只能剪掉第二个右子树的全部，或者任意一个右子树中靠右的节点。第一个右子树的第一个（靠左）子节点永远不会被剪枝。

ii. 以下面的案例为例

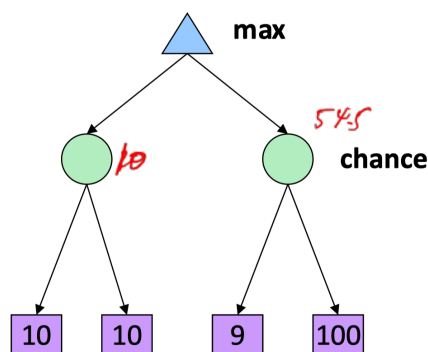


- iii. Step 1: MAX4 节点为 6, MAX5 节点为 10, MIN2 节点为 6。这个时候, 我们可以尝试改变 7 为 0, 发现 MIN2 不变, 改变 7 为 100, 发现 MIN2 不变。我们再次尝试改变 10 为 0, MIN2 不变, 改变 10 为 100, 发现 MIN2 依旧不变。这说明 7 和 10 对 MIN2 毫无影响。但是 7 属于第一个右子树的第一个 (靠左) 子节点, 因此其永远不会被剪枝。因此, 10 被剪枝。
- iv. Step 2: MAX6 节点为 3, MAX7 节点为 9, MIN3 节点为 3, MAX1 节点为 6。这个时候, 我们可以尝试改变 0 为 100, 发现 MAX1 不变, 改变 0 为 9, 发现 MAX1 不变。我们再次尝试改变 9 为 100, 发现 MAX1 不变, 改变 9 为 0, 发现 MAX 不变。这说明 0 和 9 对 MAX1 毫无影响。根据 Pruning 定义, 我们只能剪掉第二个右子树的全部, 或者任意一个右子树中靠右的节点, 第一个右子树的第一个 (靠左) 子节点永远不会被剪枝。因此, 0 和 9 被剪枝。

Expectimax Search

9. 为什么 Minimax 不够用了?
- Minimax 适用于对手是理性敌人的情况 (例如棋类)。
 - 但现实中有很多 不确定性或 “非敌人” 的情况。
 - 幽灵不总是有策略地追你。
 - 骰子随机、概率事件。

10. Expectimax Search



- a. 不是选最坏情况，而是选 **expected value** 最大的路径。我们替换掉 Min 节点 → 使用 Chance 节点。因此我们现在有
 - i. MAX 节点：取最大值，为自己的动作。
 - ii. CHANGE 节点：取加权平均值 (**expected value**)，为随机结果的可能性。
- b. **Expectimax Pruning**：在 Expectimax 中，基本不能 pruning。这是因为 Chance 节点的值是平均值。你必须访问所有子节点才能知道平均是多少，即使一个值很差，也可能被另一个高值“拉高” **expected value**。
- c. **Depth-Limited Expectimax**：和 Minimax 非常类似，我们依旧可以限制搜索深度，在叶子非 **terminal state** 下，使用 **evaluation function** 进行评分，然后像 normal expectimax 一样回传 **expected value**。

Lecture 8 Markov Decision Processes

Markov Property and Markov Decision Processes

1. Markov Property

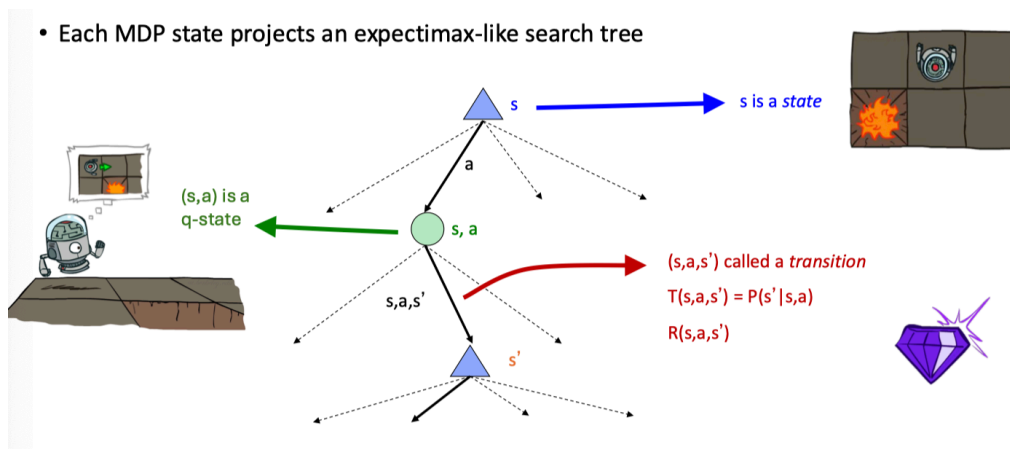
- a. Markov Property 说明已知 present state, the future and the past are independent。这是一种 memoryless 的性质。

$$P(S_{t+1} = s' \mid S_t = s_t, A_t = a_t, S_{t-1} = s_{t-1}, A_{t-1} = a_{t-1}, \dots, S_0 = s_0) \\ = P(S_{t+1} = s' \mid S_t = s_t, A_t = a_t)$$

- b. 该公式说明已知现在的 state S_t 和当前采取的行动 A_t 。下一时态的 state S_{t+1} 的分布不再依赖于更早之前的 state 和 history of actions。

2. MDP Search Tree

- Each MDP state projects an expectimax-like search tree



- a. State s : 当前所在的位置
- b. Q-state (s, a) : 在 state s 下采取 action a 的决策点，这就好像是机器人脑中有一个“前进”的计划。表示当前状态+选定动作。
- c. Transition (s, a, s') : 表达从 s 经过动作 a 到 s' 。
- d. s' : 执行 action a 后可能到达的 state s' 。
- Transition probability $T(s, a, s')$: 达到后继状态 s' 的概率。
 - Rewards $R(s, a, s')$: 即时奖励。在 state s ，我们执行 action a ，结果达到了 s' ，我们立刻收到一个 rewards。
 - Value function: 从 state s 开始，遵循 optimal strategy π^* ，在未来将会累计获得的总奖励（折扣后）。

3. Discounting

- a. 我们的目标是最大化奖励总和，并且更偏好现在的奖励而不是未来的奖励。为了将“未来奖励价值较低”这一思想建模进来，我们引入 discounting factor $\gamma \in [0, 1)$ ，让奖励的价值指数级递减。
- 当前步 discounting factor 为 $\gamma^0 = 1$ 。
 - 下一步 discounting factor 为 $\gamma^1 = \gamma$ 。

- iii. 在下一步 discounting factor 为 γ^2
- b. Why discount?
 - i. 现实性理由：早期的奖励确实更有价值。
 - ii. 计算理由：帮助算法收敛。如果不折扣，算法可能会因为未来的奖励加起来无限大而不稳定。
- c. 案例 discounting factor $\gamma = 0.5$ 。
 - i. $U([1,2,3]) = 1 \times 1 + 0.5 \times 2 + 0.25 \times 3 = 2.75$
 - ii. $U([3,2,1]) = 1 \times 3 + 0.5 \times 2 + 0.25 \times 1 = 4.25$
 - iii. 这个案例也说明早期拿到的奖励更高，折现价值更大。
- d. Stationary Preferences: 如果我们认为序列 $[a_1, a_2, \dots]$ 比 $[b_1, b_2, \dots]$ 更好，那么在两个序列前面都加上相同的初始奖励 r ，我们仍然会偏好原本更优的序列。

$$[a_1, a_2, \dots] \succ [b_1, b_2, \dots] \Leftrightarrow [r, a_1, a_2, \dots] \succ [r, b_1, b_2, \dots]$$
- e. Utility 的定义
 - i. Additive Utility: $U([r_0, r_1, r_2, \dots]) = r_0 + r_1 + r_2 + \dots$ 。每个奖励都等价地被加入总值，不考虑时间因素。
 - ii. Discounted Utility: $U([r_0, r_1, r_2, \dots]) = r_0 + \gamma r_1 + \gamma^2 r_2 + \dots$ 。体现时间偏好，也是我们在 MDP 总常用的 utility 模型。
- f. Infinite Utilities: 如果一个 agent 永远在环境中行动（例如没有终点或退出），那么它获得的奖励会不会无限大？解决方案如下：
 - i. Finite horizon: 只考虑 T 步之内的奖励。
 - ii. Discounting（最常见的方法）：令一个 γ ，让未来奖励指数衰减

$$U([r_0, r_1, r_2, \dots]) = \sum_{t=0}^{\infty} \gamma^t r_t$$

- iii. Absorbing state: 设置某个 state 为终点，一旦进入就不能离开。

Values of States and Solving MDP

4. Recursive definition of value

a. 最优 value function

$$V^*(s) = \max_a Q^*(s, a)$$

从状态 s 开始未来总价值（假设每步都选最优）。我们从所有可选动作中，挑出 Q 值最大的动作所对应的期望 utility。

b. Q 值函数

$$Q^*(s, a) = \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^*(s')]$$

在 state s 下，执行 action a ，然后接下来一直 follow optimal strategy 行动，我预计能总共赚到多少（discounting 后的）奖励。

- c. 组合公式 (Bellman Optimality Equation)

$$V^*(s) = \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^*(s')]$$

简单来讲就是分别计算针对不同 action 的 $Q^*(s, a)$ ，然后选出最大的那个作为我们的 $V^*(s)$ 值。

- d. 案例 1: 假设我们有 2 个 states s_1 和 s_2 ，action 也只有一个 a ，也就是从 s_1 执行 action a 后，有 80% 的概率转移至 s_2 ，奖励为 $R(s_1, a, s_2) = 5$ ；有 20% 的概率转移至 s_1 ，奖励为 $R(s_1, a, s_1) = 1$ 。另外 $\gamma = 0.9$ ， $V^*(s_2) = 10$ 。

- i. 对于 $s = s_1, a = a$

$$\begin{aligned} Q^*(s_1, a) &= 0.2 \cdot [1 + 0.9 \cdot V^*(s_1)] + 0.8 \cdot [5 + 0.9 \cdot V^*(s_2)] \\ &= 11.4 + 0.18V^*(s_1) \end{aligned}$$

- ii. 由于 action 只有一个，所以 $V^*(s_1) = Q^*(s_1, a) = 11.4 + 0.18V^*(s_1)$ ，解方程可得 $V^*(s_1) = 13.9$ 。

- iii. 因此， $Q^*(s_1, a) = 13.9$ 。

5. Solving MDP — Value Iteration

- a. 方法: 我们不断使用 Bellman Optimality Equation 更新每个状态的值，直到收敛

$$V_{k+1}(s) = \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V_k(s')]$$

收敛或达到最大轮数后，提取 optimal strategy。

$$\pi^*(s) = \arg \max_a Q^*(s, a)$$

- b. 案例: 假设一个小的 MDP [S1] — [S2] — [S3]。

- i. State 集合: $S = \{S1, S2, S3\}$
- ii. Action 集合: $A = \{\text{Left}, \text{Right}\}$
- iii. Discounting factor: $\gamma = 0.9$
- iv. 初始所有 $V(s) = 0$ (第 0 次迭代)
- v. S3 是终点: 奖励 $R = +1$ (其他奖励为 0)
- vi. Transition dynamics (动作具有随机性)
 - S1, Right, 0.8 到 S2, 0.2 留 S1
 - S1, Left, 1.0 留 S1
 - S2, Right, 0.9 到 S3, 0.1 回 S1
 - S2, Left, 0.9 到 S1, 0.1 到 S3

- c. Complexity of each iteration: $O(S^2A) = O(3^2 \times 2) = O(18)$

- d. $K = 0$: $V_0(S_1) = 0, V_0(S_2) = 0, V_0(S_3) = 1$ 。

- e. $K = 1$

- i. 状态 S2，我们尝试两个动作

- $Q(S2, \text{Right}) = 0.9 \cdot (0 + 0.9 \cdot V_0(S3)) + 0.1 \cdot (0 + 0.9 \cdot V_0(S1)) = 0.9 \cdot 0.9 \cdot 1 + 0.1 \cdot 0 = 0.81$
 - $Q(S2, \text{Left}) = 0.9 \cdot (0 + 0.9 \cdot V_0(S1)) + 0.1 \cdot (0 + 0.9 \cdot V_0(S3)) = 0 + 0.1 \cdot 0.9 = 0.09$
 - $V_1(S2) = \max(0.81, 0.09) = 0.81$
 - ii. 状态 S1, 我们尝试两个动作
 - $Q(S1, \text{Right}) = 0.8 \cdot (0 + 0.9 \cdot V_0(S2)) + 0.2 \cdot (0 + 0.9 \cdot V_0(S1)) = 0 + 0 = 0$
 - $Q(S1, \text{Left}) = 1.0 \cdot (0 + 0.9 \cdot V_0(S1)) = 0$
 - $V_1(S1) = \max(0, 0) = 0$
 - iii. 状态 S3, 依然为 1。
- f. $K = 2$
- i. 状态 S1, 我们尝试两个动作
 - $Q(S1, \text{Right}) = 0.8 \cdot (0 + 0.9 \cdot V_1(S2)) + 0.2 \cdot (0 + 0.9 \cdot V_1(S1)) = 0.8 \cdot (0 + 0.9 \cdot 0.81) + 0.2 \cdot (0 + 0.9 \cdot 0) = 0.8 \cdot 0.729 + 0 = 0.5832$
 - $Q(S1, \text{Left}) = 1.0 \cdot (0 + 0.9 \cdot V_1(S1)) = 0$
 - $V_2(S1) = \max(0.5832, 0) = 0.5832$
 - ii. 状态 S2, 我们尝试两个动作
 - $Q(S2, \text{Right}) = 0.9 \cdot (0 + 0.9 \cdot V_1(S3)) + 0.1 \cdot (0 + 0.9 \cdot V_1(S1)) = 0.9 \cdot 0.9 \cdot 1 + 0.1 \cdot 0 = 0.81$
 - $Q(S2, \text{Left}) = 0.9 \cdot (0 + 0.9 \cdot V_1(S1)) + 0.1 \cdot (0 + 0.9 \cdot V_1(S3)) = 0 + 0.1 \cdot 0.9 = 0.09$
 - $V_2(S2) = \max(0.81, 0.09) = 0.81$
 - iii. 状态 S3, 依然为 1。
- g. 寻找 Optimal strategy

$$\pi^*(s) = \arg \max_a Q^*(s, a)$$

对于 $K=2$, 我们已经算过 S1 和 S2 都是 Right 这个动作的 Q 值大, $\pi^*(S1) = \text{Right}$ 和 $\pi^*(S2) = \text{Right}$, 因此最优路径为 S1 --Right--> S2 --Right--> S3。

6. Solving MDP — Policy Evaluation

- a. 案例: 假设一个小的 MDP [S1] —— [S2] —— [S3]。
 - i. State 集合: $S = \{S1, S2, S3\}$
 - ii. Action 集合: $A = \{\text{Left}, \text{Right}\}$
 - iii. Discounting factor: $\gamma = 0.9$
 - iv. 初始所有 $V(s) = 0$ (第 0 次迭代)
 - v. S3 是终点: 奖励 $R = +1$ (其他奖励为 0)
 - vi. Transition dynamics (动作具有随机性)

- S1, Right, 0.8 到 S2, 0.2 留 S1
- S1, Left, 1.0 留 S1
- S2, Right, 0.9 到 S3, 0.1 回 S1
- S2, Left, 0.9 到 S1, 0.1 到 S3

b. 初始化一个随机策略 π_0

- S1: Right
- S2: Right
- S3: —

c. Policy Evaluation

- 对于 S1, 转移到 S2 的概率是 0.8, 留在 S1 的概率是 0.2。

$$\begin{aligned} V(S1) &= 0.8[0 + 0.9V(S2)] + 0.2[0 + 0.9V(S1)] \\ &= 0.72V(S2) + 0.18V(S1) = 0.878 V(S2) \end{aligned}$$

- 对于 S2, 有 0.9 概率到 S3, 有 0.1 概率回到 S1。

$$\begin{aligned} V(S2) &= 0.9[0 + 0.9 \cdot V(S3)] + 0.1[0 + 0.9 \cdot V(S1)] \\ &= 0.9 \times 0.9 \times 1 + 0.1 \times 0.9V(S1) = 0.81 + 0.09 V(S1) \end{aligned}$$

- 解方程可得, $V^{\pi_0}(S1) \approx 0.772$, $V^{\pi_0}(S2) \approx 0.879$, $V^{\pi_0}(S3) \approx 1$ 。

d. Policy Improvement

- 对于 S1, 如果选择 Right

$$Q(S1, \text{Right}) = 0.8[0 + 0.9V^{\pi_0}(S2)] + 0.2[0 + 0.9V^{\pi_0}(S1)] = 0.7719$$

如果选择 Left

$$Q(S1, \text{Left}) = 1.0 \times [0 + 0.9V^{\pi_0}(S1)] = 0.9 \times 0.772 \approx 0.6948$$

对比可得

$$Q(S1, \text{Right}) \approx 0.772 > 0.6948 \approx Q(S1, \text{Left})$$

因此, 改进后的策略在 S1 保持为 Right。

- 对于 S2, 如果选择 Right

$$\begin{aligned} Q(S2, \text{Right}) &= 0.9[0 + 0.9 \cdot V^{\pi_0}(S3)] + 0.1[0 + 0.9 \cdot V^{\pi_0}(S1)] \\ &= 0.8795 \end{aligned}$$

如果选择 Left

$$Q(S2, \text{Left}) = 0.9[0 + 0.9 \cdot V^{\pi_0}(S1)] + 0.1[0 + 0.9 \cdot V^{\pi_0}(S3)] = 0.7153$$

对比可得

$$Q(S2, \text{Right}) \approx 0.8795 > 0.7153 \approx Q(S2, \text{Left})$$

因此 S2 的最优动作也为 Right。

Lecture 9 Bayes' Nets

Representation

1. Probabilistic Models

- Model 帮助我们理解和表达 how (a portion of) the world works。但这始终是一种 simplifications。
- Model 永远是简化但有用的。世界非常复杂，model 无法囊括所有 variables 和所有 variables 之间的相互作用。All models are wrong; but some are useful.
- What do we do with probabilistic models?
 - Explanation: 例如看到症状推断病因；
 - Prediction: 例如已知天气预测交通状况；
 - Value of Information: 判断是否值得获取更多信息来做决策。

2. Independence

- 定义：两个 variables X 和 Y 被认为是 independent 的，当且仅当
$$\forall x, y: P(x, y) = P(x)P(y) \ \& \ P(x|y) = P(x)$$
我们用符号表示为 $X \perp\!\!\!\perp Y$ 。
- 案例：

$P_1(T, W)$			$P(T)$		$P_2(T, W)$		
T	W	P	T	P	T	W	P
hot	sun	0.4	hot	0.5	hot	sun	0.3
hot	rain	0.1	cold	0.5	hot	rain	0.2
cold	sun	0.2			cold	sun	0.3
cold	rain	0.3			cold	rain	0.2

$P(W)$	
W	P
sun	0.6
rain	0.4

- $P(\text{hot}, \text{sun}) = P(\text{hot}) \times P(\text{sun}) = 0.5 \times 0.6 = 0.3$
 - $P(\text{hot}, \text{rain}) = P(\text{hot}) \times P(\text{rain}) = 0.5 \times 0.4 = 0.2$
 - $P(\text{hot}, \text{sun}) = P(\text{hot}) \times P(\text{sun}) = 0.5 \times 0.6 = 0.3$
 - $P(\text{cold}, \text{rain}) = P(\text{cold}) \times P(\text{rain}) = 0.5 \times 0.4 = 0.2$
 - 因此 $P_2(T, W)$ 是正确的。
- Conditional Independence
 - 为什么 Unconditional Independence 很重要：Unconditional (absolute) independence 很罕见 —— 因为真实世界中的变量往往是相互关联的。Conditional independence 更常见、更实用，是我们理解世界的一种工具。

ii. 数学定义

$$\forall x, y, z: P(x, y|z) = P(x|z)P(y|z)$$

$$\forall x, y, z: P(x|y, z) = P(x|z)$$

如果知道 Z 的值，那么 Y 给不给你看都不影响你对 X 的判断。

iii. 案例：我们有三个 variables（toothache 牙痛、cavity 蛀牙、catch 探针卡住）

- 当已知 cavity 的时候，catch 与 toothache 条件独立。

$$P(\text{Catch} | \text{Toothache}, \text{Cavity}) = P(\text{Catch} | \text{Cavity})$$

- 这意味着如果你知道是否有蛀牙，那么是否牙痛不会影响探针是否卡住的概率。

$$P(+\text{catch} | +\text{toothache}, +\text{cavity}) = P(+\text{catch} | +\text{cavity})$$

$$P(+\text{catch} | +\text{toothache}, -\text{cavity}) = P(+\text{catch} | -\text{cavity})$$

d. Conditional Independence and the Chain Rule

i. 普通的 chain rule

$$P(X_1, X_2, \dots, X_n) = P(X_1)P(X_2|X_1)P(X_3|X_1, X_2) \dots$$

ii. 一个例子：Traffic, Rain, Umbrella

- 无任何条件独立假设时： $P(\text{Traffic}, \text{Rain}, \text{Umbrella}) = P(\text{Rain})P(\text{Traffic} | \text{Rain})P(\text{Umbrella} | \text{Rain}, \text{Traffic})$
- 做出 $\text{Umbrella} \perp \text{Traffic} | \text{Rain}$ 假设后： $P(\text{Traffic}, \text{Rain}, \text{Umbrella}) = P(\text{Rain})P(\text{Traffic} | \text{Rain})P(\text{Umbrella} | \text{Rain})$

3. Bayesian Networks

a. Bayesian Networks 解决了什么问题

- i. 我们在使用 joint distribution table 的时候，如果 variables 太多，那么表格会变的非常庞大，难以表示、存储和运算。
- ii. 很难同时对几个以上的 variables 进行学习（估计）。
- iii. Bayesian networks（正式的称为 graphical models）用图结构表示 variables 之间的相互作用关系。

b. Graphical Model Notation

- i. Node: 表示 variables，每个 variables 都有自己的 domain。比如 weather 这个 variables 的 domain 可以是 sun 和 rain。Variables 可以是：
 - assigned（已观察到）：比如你知道今天牙痛。
 - unassigned（未观察到）：比如你不知道你有蛀牙。
- ii. Arcs: 表示 variables 之间的“依赖关系/direct influence”。具体来说，arcs encode conditional independence（见后文）。但是，箭头 $\neq$ 因果关系，形式上它们只是统计依赖，不一定意味着因果性。

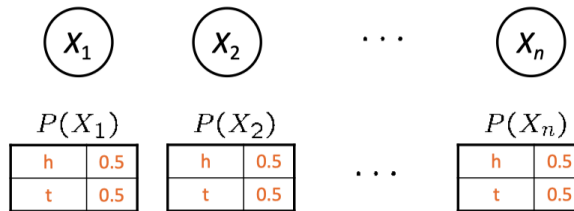
c. Bayes' Net Semantics

- i. Nodes
- ii. DAG (有向无环图 Directed Acyclic Graph): 节点通过箭头连接, 箭头表示依赖关系。
- iii. Conditional Probability Table (CPT): Bayes' nets implicitly encode joint distributions

$$P(x_1, x_2, \dots, x_n) = \prod_{i=1}^n P(x_i \mid \text{parents}(X_i))$$

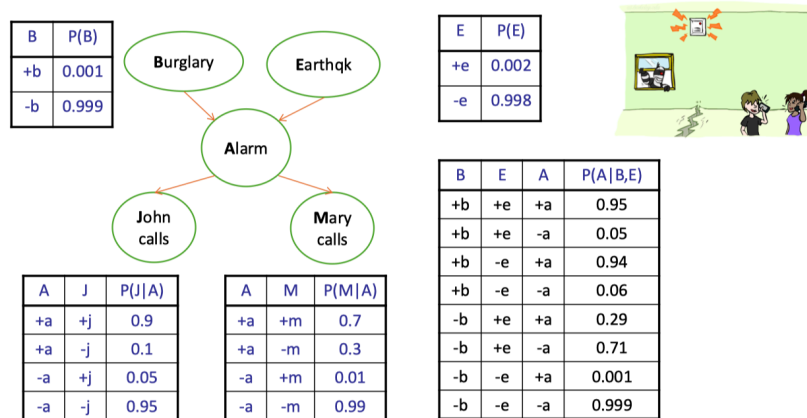
对于每一个 variable x_i , 我们只考了其在父节点给定下的条件概率。
最终乘起来就得到整个网络中这个变量组合的 joint distribution。

d. 案例一



- i. Node 为 $X_1, X_2, \dots, X_n$ 。
- ii. DAG 已经给出。我们发现没有一个 arcs, 这说明所有变量之间都完全独立。
- iii. CPT: 根据公式, 由于每个 node 之间没有连线, 因此不存在父节点, 这说明 $P(X_1, X_2, X_3, \dots, X_n) = P(X_1)P(X_2) \dots P(X_n)$, 因此, 我们可以得出一些结论
 - $P(h, h, t, h) = P(h)P(h)P(t)P(h) = 0.5^4 = 0.0625$
 - $P(h, h, h) = P(h)P(h)P(h) = 0.5^3 = 0.125$

e. 案例二



- i. Node 为 B, E, A, J, M 。

- ii. DAG 已经给出，我们发现了有四个 arcs。其中 B 和 E 为父节点，A 为 B 和 E 的子节点，J 和 M 为 A 的子节点。
- iii. CPT: $P(B, E, A, J, M) = P(B)P(E)P(A|B, E)P(J|A)P(M|A)$ 。因此，我们可以得出一些结论
 - $P(+b, -e, +a, +j, -m) = P(+b) \cdot P(-e) \cdot P(+a | +b, -e) \cdot P(+j | +a) \cdot P(-m | +a) = 0.000253$
- f. Bayes' Net 在表示联合概率分布时的空间效率优势
 - i. 完整联合概率：对于 N 个 Boolean variables，完整联合概率的大小为 2^N ，因为每个 Boolean variables 有 2 个取值 (true/false)，所以联合分布有 2^N 个可能的 state 组合。
 - ii. N-node Net: 如果有 k 个父节点，大小为 $O(N \cdot 2^{k+1})$ 。
 - iii. 这再次说明了 Bayes' Net 的优势
 - Huge space savings
 - Easier to elicit local CPTs
 - Faster to answer queries

Conditional Independences

4. Local Markov Property

- a. 定义：对于每一个节点 X_i ，它与它的非后代节点在给定它的父节点的条件下是 conditional independent。

$$X_i \perp \text{NonDescendants}(X_i) \mid \text{Parents}(X_i)$$

- b. 示例：结构 $X \rightarrow Y \rightarrow Z \rightarrow W$ 。
 - i. 对于节点 Z
 - 父节点：Y
 - 非后代（且非父节点）：X
 - 得出： $Z \perp X \mid Y$
 - ii. 对于节点 W
 - 父节点：Z
 - 非后代（且非父节点）：X、Y
 - 得出： $W \perp X \mid Z, W \perp Y \mid Z$

5. D-Separation（更全面的判断 Conditional Independences 的方法）

a. Active Paths and Inactive Paths

- i. 一条路径由多个 triples 注册。一条路径只有当其中每一个 triple 都是 active 的时候，整条路径才是 active 的。
 - Causal Chain: $A \rightarrow B \rightarrow C$ 或 $A \leftarrow B \leftarrow C$ （条件为 B 不被观察）
 - Common Cause: $A \leftarrow B \rightarrow C$ （条件为 B 不被观察）

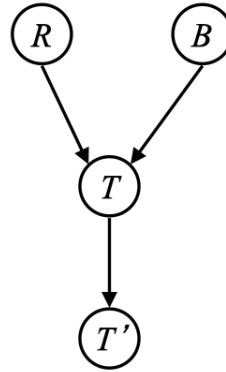
- Common Effect (V-structure): $A \rightarrow B \leftarrow C$ (B 被观察 或 B 的后代被观察)

ii. 被观察是指对应的 variable 是被 given that 的。

iii. 只要路径上有一个 triple 是 inactive 的，这条路径就被 blocked。

b. D-Separation 原则：考虑 X 和 Y 之间所有的路径，如果所有路径都被 blocked，那么 X 和 Y conditional independent。

c. 案例一

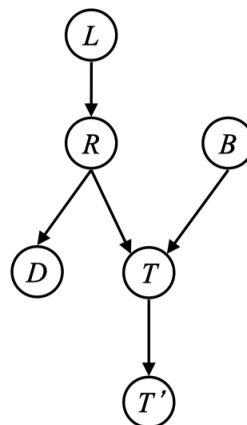


i. R 和 B 独立：R 和 B 构成 $R \rightarrow T \leftarrow B$ 路径，但 T 没有被观察（没有 given T），因此这条路径被 blocked，因此 R 和 B 独立。

ii. R 和 B Given T 不独立：R 和 B 构成 $R \rightarrow T \leftarrow B$ 路径，T 被观察，因此这条路径为 active，因此 R 和 B Given T 不独立。

iii. R 和 B Given T' 不独立：R 和 B 构成 $R \rightarrow T \leftarrow B$ 路径，T'（T 的后代）被观察，因此这条路径为 active，因此 R 和 B Given T' 不独立。

d. 案例二



i. L 和 T' Given T 独立：L 和 T' 构成 $L \rightarrow R \rightarrow T \rightarrow T'$ 路径，可以分为 $L \rightarrow R \rightarrow T$ 和 $R \rightarrow T \rightarrow T'$ 两个子路径。

- $L \rightarrow R \rightarrow T$: R 没有被观察，Active，不 Blocked。

- $R \rightarrow T \rightarrow T'$: T 被观察, 不 Active, Blocked。
- 因此 $L \rightarrow R \rightarrow T \rightarrow T'$ 被 blocked, L 和 T' Given T 独立。
- ii. L 和 B 独立: L 和 B 构成 $L \rightarrow R \rightarrow T \leftarrow B$ 路径, 可以分为 $L \rightarrow R \rightarrow T$ 和 $R \rightarrow T \leftarrow B$ 两个子路径。
- $L \rightarrow R \rightarrow T$: R 没有被观察, Active, 不 Blocked。
 - $R \rightarrow T \leftarrow B$: T 没有被观察, 不 Active, Blocked。
- 因此 $L \rightarrow R \rightarrow T \leftarrow B$ 被 blocked, L 和 B 独立。
- iii. L 和 B Given T/R 独立: L 和 B 构成 $L \rightarrow R \rightarrow T \leftarrow B$ 路径, 可以分为 $L \rightarrow R \rightarrow T$ 和 $R \rightarrow T \leftarrow B$ 两个子路径。
- $L \rightarrow R \rightarrow T$: R 被观察, 不 Active, Blocked。
 - $R \rightarrow T \leftarrow B$: T 被观察, Active, 不 Blocked。
- 因此 $L \rightarrow R \rightarrow T \leftarrow B$ 被 blocked, L 和 B Given T/R 独立。

Inference

6. 本节案例

- a. $R \rightarrow T \rightarrow L$ (R 是否下雨; T 是否有交通堵塞; L 是否迟到)
- b. $P(R): P(+r) = 0.1, P(-r) = 0.9$
- c. $P(T|R)$:

R	+t	-t
+r	0.8	0.2
-r	0.1	0.9

- d. $P(L|T)$:

T	+l	-l
+t	0.3	0.7
-t	0.1	0.9

- e. 我们的目标是计算 $P(L = +l)$ 。

7. Inference by Enumeration

- a. Step 1: 根据 DAG 图形, 我们先写出联合分布

$$P(R, T, L) = P(R)P(T|R)P(L|T)$$

- b. Step 2: 枚举所有 variable 取值。我们要求 $P(L = +l)$, 因此需要对 R 和 T 求和。

$$P(+l) = \sum_{r \in \{+r, -r\}} \sum_{t \in \{+t, -t\}} P(r) \cdot P(t|r) \cdot P(+l|t)$$

i. Case 1: $R = +r, T = +t$

$$P(+r) \cdot P(+t|+r) \cdot P(+l|+t) = 0.1 \cdot 0.8 \cdot 0.3 = 0.024$$

ii. Case 2: $R = +r, T = -t$

$$P(+r) \cdot P(-t|+r) \cdot P(+l|-t) = 0.1 \cdot 0.2 \cdot 0.1 = 0.002$$

iii. Case 3: $R = -r, T = +t$

$$P(-r) \cdot P(+t|-r) \cdot P(+l|+t) = 0.9 \cdot 0.1 \cdot 0.3 = 0.027$$

iv. Case 4: $R = -r, T = -t$

$$P(-r) \cdot P(-t|-r) \cdot P(+l|-t) = 0.9 \cdot 0.9 \cdot 0.1 = 0.081$$

c. Step 3: 对所有情况求和。

$$P(+l) = 0.024 + 0.002 + 0.027 + 0.081 = 0.134$$

因此，迟到的概率为 0.134（即 13.4%）。

8. Variable Elimination

a. Step 1: 根据 DAG 图形，我们先写出联合分布

$$P(R, T, L) = P(R)P(T|R)P(L|T)$$

我们将联合分布的组成部分作为 Factors。

i. Factor 1: $f_1(R) = P(R)$

ii. Factor 2: $f_2(T, R) = P(T|R)$

iii. Factor 3: $f_3(L, T) = P(L|T)$

b. Step 2: 消除第一个隐藏变量（我们的目标变量是 L，第一个需要消除的隐藏变量是 R）。

i. 我们现在有两个 factors 涉及 R: $P(R), P(T|R)$ 。我们将其相乘，可以得到第四个 factor

$$f_4(T, R) = P(R) \cdot P(T|R)$$

ii. 计算 $f_4(T, R)$ 的结果

$$f_4(+t, +r) = P(+r) \cdot P(+t|+r) = 0.1 \times 0.8 = 0.08$$

$$f_4(-t, +r) = P(+r) \cdot P(-t|+r) = 0.1 \times 0.2 = 0.02$$

$$f_4(+t, -r) = P(-r) \cdot P(+t|-r) = 0.9 \times 0.1 = 0.09$$

$$f_4(-t, -r) = P(-r) \cdot P(-t|-r) = 0.9 \times 0.9 = 0.81$$

iii. 我们对 $f_4(T, R)$ 中的 R 进行 marginalize，也就是把所有 R 的取值想家，得到一个只涉及 T 的因子

$$f_5(T) = \sum_R f_4(T, R)$$

- 对于 $T = +t$: $f_5(+t) = f_4(+t, +r) + f_4(+t, -r) = 0.08 + 0.09 = 0.17$

- 对于 $T = -t$: $f_5(-t) = f_4(-t, +r) + f_4(-t, -r) = 0.02 + 0.81 = 0.83$

c. Step 3: 消除第二个隐藏变量（我们的目标变量是 L，第二个需要消除的隐藏变量是 T）。

- 我们现在有两个 factors 涉及 T: $f_5(T), f_3(L, T)$ 。我们将其相乘，可以得到第六个 factor

$$f_6(L, T) = f_5(T) \cdot f_3(L, T)$$

- 计算 $f_6(L, T)$ 的结果

$$f_6(+l, +t) = f_5(+t) \times P(+l | +t) = 0.17 \times 0.3 = 0.051$$

$$f_6(-l, +t) = f_5(+t) \times P(-l | +t) = 0.17 \times 0.7 = 0.119$$

$$f_6(+l, -t) = f_5(-t) \times P(+l | -t) = 0.83 \times 0.1 = 0.083$$

$$f_6(-l, -t) = f_5(-t) \times P(-l | -t) = 0.83 \times 0.9 = 0.747$$

- 我们对 $f_6(L, T)$ 中的 T 进行 marginalize，也就是把所有 T 的取值相加，得到一个只涉及 L 的因子

$$f_7(L) = \sum_T f_6(L, T)$$

- 对于 $L = +l$, $f_7(+l) = f_6(+l, +t) + f_6(+l, -t) = 0.051 + 0.083 = 0.134$

- 对于 $L = -l$, $f_7(-l) = f_6(-l, +t) + f_6(-l, -t) = 0.119 + 0.747 = 0.866$

d. $P(+l) = 0.134$

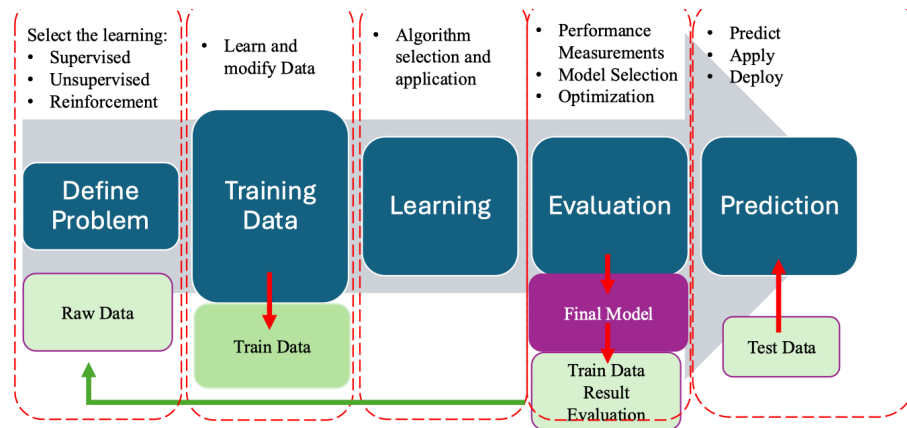
Lecture 10 Machine Learning

Introduction to Machine Learning

1. ML is everywhere: Computer Science、Healthcare、Retail、Manufacturing、Energy、Financial Service。
2. 什么是 Machine Learning
 - a. 历史: 术语 “Machine Learning” 最早由 IBM 的 Arthur Samuel 在 1959 年提出。
 - b. 核心目标: It focused on designing algorithms that recognize input data (train data) patterns and predict the target.
 - c. Machine Learning Algorithms (Learners)
 - i. Take advantage of data to capture characteristics of interest of their unknown underlying probability distribution (pattern).
 - ii. Use the learned pattern and test on new data (output) to predict the outcome.
 - d. One Fundamental Challenge in ML (Generalization): 如何确保模型能在未见数据上保持良好表现, 而不是仅仅记住训练数据。
3. Learning Types
 - a. Supervised Learning
 - i. 使用带标签 (labeled) 数据, 即每个输入样本都有对应的正确输出。
 - ii. 模型根据直接反馈进行学习 (例如: 预测错了, 会知道正确答案)。
 - iii. 目标是预测结果。可用于未来预测 (如股票价格、疾病风险等)。
 - iv. 分为 Classification 和 Regression 这大类。
 - b. Unsupervised Learning
 - i. 无标签或目标, 输入数据中没有明确的输出。
 - ii. 无反馈, 模型不 “知道” 是否做得对。
 - iii. 常用于降维 (如 PCA)、模式识别、推荐系统。
 - iv. 分为 Clustering (聚类) 和 Dimensionality Reduction (原数据像是一大堆复杂但冗余的信息, 降维就像是提炼出核心信息, 就像从一段长文章中抽出关键词或主旨句)。
 - c. Reinforcement Learning
 - i. 学习如何在决策过程中采取行动。
 - ii. 通过与环境互动获取奖励反馈 (如游戏胜负、机器人能否避障)。
 - iii. 学习一系列动作策略, 以获得长期回报 (长期最优策略)。
4. ML Workflow
 - a. ML 基本工作流程: Data + Output $\rightarrow$ Computer $\rightarrow$ Model
 - i. Data (输入数据): 原始 features 或观察值。

- ii. **Output**（目标变量）：或者叫 **label**，用于 supervised learning 的训练目标。比如图片是猫还是狗，房价是多少等等。
- iii. **Computer**（计算机）：训练算法的地方。
- iv. **Model**：训练好的模型。
- b. 在真实的 ML 训练流程中，我们除了训练模型之外，还需要这两步
 - i. **Evaluation / Objective Function**:
 - **Evaluation Function**：我们需要衡量模型好坏，比如准确率、F1 分数。在模型训练结束后，用于评价模型的好坏。
 - **Objective Function**：在模型训练中，用于指导优化过程，是我们下一步 optimizer 优化器需要最大化或者最小化的函数。比如我们可以使用 MSE、交叉熵、hinge loss、正则项等。
 - ii. **Optimization**：有了目标函数后，需要一种方法来不断改进模型性能，常见方法有 Gradient Descent、牛顿法等等。

c. 完整的 ML Workflow



- i. **Define Problem**
 - 输入：Raw Data（原始数据）
 - 动作：明确任务类型，如分类、回归、clustering 还是 reinforcement learning。
 - 决策：选择学习方法（supervised、unsupervised、reinforcement）。
- ii. **Training Data**
 - 从 raw data 中提取出结构化的 train data。
 - 包括数据清洗、特征工程、标签处理（如果是 supervised learning）。
- iii. **Learning**
 - 选择并应用合适的算法（比如决策树、神经网络等）。
 - 模型读取训练数据进行参数拟合。

- 输出“初步模型”。

iv. Evaluation

- 使用 **train dataset** 和验证结果评估模型表现。
- 指标包括准确率、F1 分数、损失值等。
- 目标是模型选择与优化（比如调整超参数、选择更好的结构）。

v. Prediction

- 使用训练好的模型对 **Test Data** 进行推理。
- 模型会输出预测结果，用于实际应用、部署系统或决策支持。

d. Garbage In, Garbage Out

- Garbage In, Garbage Out:** 如果你给模型喂的数据质量很差（例如：错误、缺失、不一致、噪声多、不具代表性等），那么即使使用最先进的模型、训练过程再复杂或优化再好，最终得到的模型仍然不会有实用价值，预测效果也不会理想。
- 因此我们要进行“前期准备四件套”
 - **EDA (Exploratory Data Analysis)** 探索性数据分析：**了解数据结构、分布、缺失、异常值等情况**；可视化（如直方图、散点图）、描述性统计量；帮你发现潜在问题并指导后续处理。
 - **Preprocessing (预处理)**：清洗数据、标准化/归一化、处理缺失值、编码分类变量等。
 - **Feature Engineering (特征构造)**：根据已有特征构造新的、更有信息量的特征。比如从出生日期生成年龄，比如用体重和身高算出 BMI。
 - **Feature Selection / Extraction (特征选择/提取)**：挑出最有用的特征，去掉冗余或无关变量。

e. Generalization

- ML 的意义是预测新数据：ML is an algorithm that learns a model from the input data so that the model can be used to predict certain properties of **new data**. 训练模型不是终点，能泛化到新数据并持续改进才是机器学习的精髓。
 - 我们之前流程的 1-4 步：**train model**，其从已有的输入数据（Data）中学习，构建出一个可以进行预测的模型。Data → ML Algorithm → Train Model → Prediction。
 - 我们之前流程的第 5 步：**inference**，将训练好的模型用于 New Data 上进行预测。这是 **generalization** 能力的体现——在没见过的数据上也能输出合理预测。

- ii. 为了 generalization, 我们要划分 train dataset 和 test dataset。如果不做划分 (即只训练、没有测试), 模型泛化能力根本无法评估, 直到有了未来的 “新数据”, 这显然太迟。
 - Train dataset: 用于训练模型。
 - Test dataset: 用于评估模型在未见过数据上的表现, 即衡量它的泛化能力。
 - 划分比例: 7:3 (70% 训练 / 30% 测试); 8:2、9:1 等也很常见; 比例选择会根据数据规模与任务复杂度有所调整。
- iii. 关于划分 train dataset 和 test dataset 的 facts
 - 即使训练集变小, 测试集也很重要: 用 70% 数据训练虽然 “少学了一点”, 但通过 30% 数据进行测试, 可以更好地了解模型是否过拟合, 并且调整模型 complexity/flexibility。
 - 训练集通常要大于测试集: 虽然测试集重要, 但主要目标仍是让模型能学得足够好。
 - 总的来说: 我们牺牲一点学习样本量, 换来对泛化能力的可控与评估。
- iv. 最好的数据划分方法: Holdout Cross-Validation
 - 我们将数据集分为: Training set - 用于模型训练; Validation set - 用于训练过程中调参或选择模型结构; Test set - 用于最终评估模型泛化能力。
 - a. Training dataset path: 60% 的数据用于模型拟合;
 - b. Validation dataset path: 10% 的数据在训练过程中反复评估模型效果;
 - c. Test dataset path: 30% 的数据在模型最终定型后一次性评估其泛化表现。
 - Validation dataset 的作用: Validation results are used to drive the continuation of the training process。
 - a. 模型调参 (如学习率、正则化强度)
 - b. 模型选择 (如 SVM vs. 决策树)
 - c. 早停 (early stopping) 等训练控制策略
 - Validation dataset 的缺点: 如果数据划分不合理, validation dataset 不能代表整体分布, 可能会导致选出的模型在 test dataset 上 high variance。尤其在数据量较小的情况下, 这种敏感性影响更大。为了解决划分敏感问题, 实际中经常使用 K-fold cross-validation (K 折交叉验证), 对不同划分做多次验证, 降低评估方差。

Classification: Naïve Bayes Classifier

5. Naïve Bayes Classifier 的定义

- a. Naïve Bayes Classifier 基于 Bayes' theorem, 是一种 probabilistic supervised learning algorithms。通过计算不同类别的后验概率, 选取概率最大的类别作为预测结果。我们有一些 assumptions:
 - i. Features are *conditionally independent* given the target
 - 给定 labels 后, 所有 features 彼此独立。
 - 实际上这个假设在多数真实场景下不成立, 但仍能取得不错效果。
 - ii. Linearly separable: 模型本质上是线性分类器。
 - iii. Same variance for all classes.
 - 特别是在高斯朴素贝叶斯中, 假设每类的条件概率服从相同方差的正态分布。
 - 这使得模型计算更简单 (如简化为线性判别函数)。
- b. Naïve Bayes Classifier 的优点和缺点
 - i. 优点: 训练快, 预测快 (假设虽然朴素, 但在某些场景中表现甚至优于复杂模型, 如 SVM、Logistic 回归)。
 - ii. 缺点:
 - 假设太强, 不现实。
 - 如果 test dataset 中出现 train dataset 未见的新类别或特征组合, 模型表现会明显下降。
 - 类别不平衡或类别数不同时, 贝叶斯概率计算会更复杂。

6. Naïve Bayes Classifier 公式与原理

- a. Input
 - i. $\mathbf{x}^{1 \times D}$: 表示一个具有 D 个 features 的输入样本向量 (如一个图片提取的 D 维特征)。
 - ii. C_k : 离散型 labels, 一共有 K 类, 例如 $k = 1, 2, \dots, K$, 表示不同的分类结果。
- b. 我们的目标: $P(C_k | \mathbf{x})$ 由于我们现在是一个 classification 问题, 我们的 $\mathbf{x}$ 是 features, 我们的 C 是 label, 因此我们关心的是给定这些 features 的情况下, 是哪一类 (label 是什么)。
- c. Bayes 公式回顾

$$P(C_k | \mathbf{x}) = \frac{P(\mathbf{x} | C_k) \cdot P(C_k)}{P(\mathbf{x})}$$

- i. 左侧 $P(C_k | \mathbf{x})$: Posterior, 我们关心的目标, 是给定输入 $\mathbf{x}$ 的情况下, 其属于类别 C_k 的概率。

- ii. 右侧 $P(\mathbf{x} | C_k)$: Likelihood, 类别 C_k 生成该输入的可能性。
- iii. 右侧 $P(C_k)$: Prior, 类别本身的概率（比如狗的样本占比）。
- iv. 分母 $P(\mathbf{x})$: 输入 $\mathbf{x}$ 出现在整个数据分布中的概率，考虑了它可能属于所有类别的情况的加权平均。

$$P(\mathbf{x}) = \sum_{k=1}^K P(\mathbf{x} | C_k)P(C_k)$$

7. 如何利用“条件独立性假设”将后验概率分解成多个更易计算的部分

a. 案例:

Class	(x_1, x_2)	$p(C_k)$	μ_k
1	(1,2), (2,3), (3,4.9)	0.5	(2, 3.3)
0	(2,1), (3,2), (4, 3.9)	0.5	(3, 2.3)

i. Step 1: 计算协方差矩阵

$$X_1 = \begin{bmatrix} 1 & 2 \\ 2 & 3 \\ 3 & 4.9 \end{bmatrix}$$

$$A_1 = X_1 - \mu_1 = \begin{bmatrix} 1 & 2 \\ 2 & 3 \\ 3 & 4.9 \end{bmatrix} - \mu_1 = \begin{bmatrix} 1-2 & 2-3.3 \\ 2-2 & 3-3.3 \\ 3-2 & 4.9-3.3 \end{bmatrix} = \begin{bmatrix} -1 & -1.3 \\ 0 & -0.3 \\ 1 & 1.6 \end{bmatrix}$$

$$\Sigma_1 = \frac{1}{3} A_1^T A_1 = \frac{1}{3} \begin{bmatrix} -1 & 0 & 1 \\ -1.3 & -0.3 & 1.6 \end{bmatrix} \begin{bmatrix} -1 & -1.3 \\ 0 & -0.3 \\ 1 & 1.6 \end{bmatrix} = \begin{bmatrix} 0.667 & 0.967 \\ 0.967 & 1.467 \end{bmatrix}$$

$$X_2 = \begin{bmatrix} 2 & 1 \\ 3 & 2 \\ 4 & 3.9 \end{bmatrix}$$

$$A_2 = X_2 - \mu_2 = \begin{bmatrix} 2 & 1 \\ 3 & 2 \\ 4 & 3.9 \end{bmatrix} - \mu_2 = \begin{bmatrix} 2-3 & 1-2.3 \\ 3-3 & 2-2.3 \\ 4-3 & 3.9-2.3 \end{bmatrix} = \begin{bmatrix} -1 & -1.3 \\ 0 & -0.3 \\ 1 & 1.6 \end{bmatrix}$$

$$\Sigma_1 = \Sigma_2 = \begin{bmatrix} 0.667 & 0.967 \\ 0.967 & 1.467 \end{bmatrix}$$

ii. Step 2: 计算权重向量 w

$$\begin{aligned} w^T &= (\mu_1^T - \mu_0^T) \Sigma^{-1} = \left(\begin{bmatrix} 2 \\ 3.3 \end{bmatrix}^T - \begin{bmatrix} 3 \\ 2.3 \end{bmatrix}^T \right) \Sigma^{-1} \\ &= [-1, 1] \begin{bmatrix} 33.79 & -22.28 \\ -22.28 & 15.37 \end{bmatrix} = [-56.0829, 37.6498] \end{aligned}$$

iii. Step 3: 计算偏置项 w_0

$$w_0 = \frac{1}{2} (\mu_0^T \Sigma^{-1} \mu_0 - \mu_1^T \Sigma^{-1} \mu_1) + \ln \left(\frac{\pi_1}{\pi_0} \right)$$

$$\mu_0^T \Sigma^{-1} \mu_0 = \begin{bmatrix} 3 & 2.3 \end{bmatrix} \cdot \begin{bmatrix} 33.79 & -22.28 \\ -22.28 & 15.37 \end{bmatrix} \cdot \begin{bmatrix} 3 \\ 2.3 \end{bmatrix} = 77.96$$

$$\mu_1^T \Sigma^{-1} \mu_1 = \begin{bmatrix} 2 & 3.3 \end{bmatrix} \cdot \begin{bmatrix} 33.79 & -22.28 \\ -22.28 & 15.37 \end{bmatrix} \cdot \begin{bmatrix} 2 \\ 3.3 \end{bmatrix} = 8.45$$

$$w_0 = \frac{1}{2}(77.96 - 8.45) + \ln\left(\frac{0.5}{0.5}\right) = 34.755$$

iv. Step 4: 分类函数 $h(x)$

$$\begin{aligned} h(x) &= w^T x + w_0 = w_1 x_1 + w_2 x_2 + w_0 \\ &= -56.0829x_1 + 37.6498x_2 + 34.755 \end{aligned}$$

v. Step 5: 最终预测

- 基于任何输入样本 (x_1, x_2) , 用 sigmoid 计算概率

$$P(C_1 | \mathbf{x}) = \sigma(h(\mathbf{x})) = \frac{1}{1 + \exp(-h(\mathbf{x}))}$$

- 决策

$$\hat{y} = \begin{cases} 1 & \text{if } \sigma(h(\mathbf{x})) \geq 0.5 \\ 0 & \text{otherwise} \end{cases}$$

Classification: Perceptron

8. Perceptron 的定义

- Perceptron 是一种经典的二分类算法，用于解决简单的线性可分问题。我们的目标是学习一个线性决策边界（或称超平面，hyperplane）来将数据点按类别分开。
- Perceptron 的线性模型形式

$$h(x_i) = \sum_{j=1}^D w_j x_{ij} = w_0 + w_1 x_{i1} + \dots + x_D x_{iD} = w^T x_i$$

其中 x_i 是第 i 个样本的特征向量（ D 维）； w 是 weight vector，其 sets the direction of the hyperplane； w_0 是偏置项，控制超平面在空间中的位置。具体而言，对于一条直线，slope 为 w ，intercept 为 w_0 。

- Decision Activation Function 是 Step Function

$$\hat{y}(x_i) = f(w^T x_i) = \begin{cases} 1 & \text{if } w^T x_i \geq 0 \\ -1 & \text{otherwise} \end{cases}$$

即，如果 $h(x_i)$ 大于等于 0，判为正类，否则为负类。

9. Error Function

- Perceptron 的学习目标：The learning is motivated by error function minimization。Perceptron 的核心思想是通过最小化一个 loss function 来优化模型参数 w ，使模型尽可能正确地分类所有训练样本。
- Perceptron 对错误非常敏感：由于 Perceptron 的激活函数是 step function，模型对 w 的变化非常敏感，小的改变就可能改变整个分类结果。因此 Perceptron 使用了 without shuffling 的 Stochastic Gradient Descent。
- Perceptron Criterion

$$E_p(\mathbf{w}) = - \sum_{n \in \mathcal{M}} h(\mathbf{x}_n) y_n = - \sum_{n \in \mathcal{M}} \mathbf{w}^\top \mathbf{x}_n y_n$$

$\mathcal{M}$: 表示所有被错误分类的样本集合。 $h(\mathbf{x}_n) = \mathbf{w}^\top \mathbf{x}_n$: 是超平面输出值。

$y_n \in \{-1, +1\}$: 是样本的标签。如果某个样本被误分类, 则 $y_n \cdot \mathbf{w}^\top \mathbf{x}_n < 0$ 。

d. 如何更新 $\mathbf{w}$

i. 如果分类正确: $\mathbf{w}^{(\tau+1)} = \mathbf{w}^{(\tau)}$, 也就是不做更新。

ii. 分类错误时如何更新: $\mathbf{w}^{(\tau+1)} = \mathbf{w}^{(\tau)} + \eta y_n \mathbf{x}_n$, 其中 η 为学习率 learning rate。

10. 案例: 此案例中 learning rate 为 1。

i	1	2	3	4	5	6
$\mathbf{x}$	(1,2)	(2,3)	(3,4.9)	(2,1)	(3,2)	(4,3.9)
y	1	1	1	-1	-1	-1

a. Step 1: 设 initial weights 为 $\mathbf{w}^0 = [5, -6, 1]$, 也就是说我们现在公式为 $y = 5 - 6x_1 + 1x_2$ 。

b. Step 2: 对于样本[1,1,2], 其 label 是 1。

$$h = \mathbf{w}^\top \tilde{\mathbf{x}} = 5 + (-6) \cdot 1 + 1 \cdot 2 = 5 - 6 + 2 = 1$$

$$\hat{y} = \text{sign}(h) = \text{sign}(1) = 1$$

正确, 不更新权重。

c. Step 3: 对于样本[1,2,3], 其 label 是 1。

$$h = 5 + (-6) \cdot 2 + 1 \cdot 3 = 5 - 12 + 3 = -4 \Rightarrow \hat{y} = -1 \neq y$$

其错误分类了, 因此需要执行更新, 在这里, 我们针对的样本是[1,2,3], 其 label 是 1, 根据公式 $\mathbf{w}^{(\tau+1)} = \mathbf{w}^{(\tau)} + \eta y_n \mathbf{x}_n$ 。

$$\mathbf{w}^{(1)} = \mathbf{w}^{(0)} + \eta \cdot y \cdot \tilde{\mathbf{x}} = [5, -6, 1] + 1 \cdot 1 \cdot [1, 2, 3] = [6, -4, 4]$$

d. Step 4: 对于样本[1,3,4.9], 其 label 是 1。

$$h = 6 + (-4) \cdot 3 + 4 \cdot 4.9 = 6 - 12 + 19.6 = 13.6 \Rightarrow \hat{y} = 1$$

正确, 不更新权重。

e. Step 5: 对于样本[1,2,1], 其 label 为-1。

$$h = 6 + (-4) \cdot 2 + 4 \cdot 1 = 6 - 8 + 4 = 2 \Rightarrow \hat{y} = 1 \neq -1$$

其错误分类了, 因此需要执行更新, 在这里, 样本是[1,2,1], 其 label 为-1, 根据公式 $\mathbf{w}^{(\tau+1)} = \mathbf{w}^{(\tau)} + \eta y_n \mathbf{x}_n$ 。

$$\mathbf{w}^{(2)} = [6, -4, 4] + (-1) \cdot [1, 2, 1] = [5, -6, 3]$$

f. 以此类推。

Classification: Supportive Vector Machine (SVM)

11. Motivation

a. Perceptron 的缺点: 最终找到的 hyperplane 强烈依赖于初始参数的选择 (如初始 weighting factor)。即便训练数据可以完美线性分类, 也有可能存在多

个 hyperplane 能做到这一点。Perceptron 不会告诉你哪一个“更好”，它只要能收敛就算完成任务了。

- b. 我们想要的是
 - i. Smallest generalization error
 - 意味着模型在面对新数据时也能保持良好表现（即“鲁棒性强”、“适用性广”）。
 - 感知机虽然能分类训练数据，但不保证对测试数据也有效。
 - ii. Similar distances between points to hyperplane
 - 有助于提高模型对数据扰动的容忍度，避免过拟合。

12. Support Vector Machine (SVM)

- a. 定义和目标：在 feature space 中找到一个 hyperplane，使得该平面到最近数据点的距离最大。这一目标直接对应 SVM 的几何解释：最大化 margin，提高泛化能力。
- b. 在 linear space 中，SVM 是 parametric model $h(x) = w_0 + w_1x_1 + w_2x_2$ 。
- c. 在 nonlinear space 中，SVM 是 non-parametric model，利用 kernel trick，把原始输入空间映射到高维空间，使得非线性可分问题变为线性可分。
- d. 和其他模型的对比
 - i. NB: independence, linear representation, Gaussian: 这些是前面 Naive Bayes (NB) 模型的假设，比如特征独立性、线性化处理、先验为高斯。
 - ii. Perceptron: linear separable: 感知机依赖于线性可分假设，不能处理非线性分类问题。
- e. How does SVM work?
 - i. Maximum Margin: SVM 寻找一个决策边界 (hyperplane)，使得它与训练数据中最近的点之间的垂直距离最大（即最大间隔）。
 - ii. Globally Optimized: SVM 解的是一个 convex optimization，保证得到的是全局最优解。与此对比，perceptron 只能得到一个局部收敛的解，可能依赖初始化，不一定最优。

13. 线性支持向量机 Linear SVM

- a. Main Concepts
 - i. Decision Boundary: The SVM learns a hyperplane. 与 Perceptron 和 Naive Bayes 的一个区别，它不依赖概率建模，而是直接从几何结构中寻找最优分隔面。
 - ii. Margin: 是 decision boundary 和 support vectors 的最小垂直距离。SVM 的目标就是最大化 margin，提高模型对新样本的 robustness。

iii. Support Vectors: 距离 decision boundary 上最近的点, 只有 support vectors 会影响最终模型, 其他点不会。

b. SVM 公式

- i. 目标: 在所有能正确分类训练样本的超平面中, 选出 “margin 最大” 的那一个, 提升模型的泛化能力。
- ii. 原始形式模型: 我们定义模型为

$$h(x) = w^T x + w_0$$

- Optimization goals: $\min_w \frac{1}{2} \|w\|^2$
- Constraints (correctly categorized and satisfying margin ≥ 1)

$$y_i(w^T x_i + w_0) \geq 1$$
- The solution for w and w_0 is found that satisfies

$$\hat{y}_i = \begin{cases} +1 & \text{if } h(\mathbf{x}_i) > 0 \\ -1 & \text{if } h(\mathbf{x}_i) < 0 \end{cases}$$

iii. 如何计算 weighting factor (Lagrange Multipliers in SVM)

$$\mathcal{L}(\mathbf{w}, w_0, \alpha) = \frac{1}{2} \|\mathbf{w}\|^2 - \sum_{i=1}^N \alpha_i [y_i(\mathbf{w}^T \mathbf{x}_i + w_0) - 1]$$

- Weighting Vector 表达式 $\mathbf{w} = \sum_{i=1}^N \alpha_i y_i \mathbf{x}_i$ (我们为了方便也可以设置 $\alpha_i = 1$, α_i 被称作 Lagrange Multipliers)
- Lagrange Multipliers 约束条件: $\sum_{i=1}^N \alpha_i y_i = 0$

iv. 如何计算偏置 Bias w_0

$$w_0 = y_i - \mathbf{w}^T \mathbf{x}_i$$

在实际应用中, 通常对多个支持向量求平均, 得到:

$$w_0 = \frac{1}{N} \sum_i (y_i - \mathbf{w}^T \mathbf{x}_i)$$

c. 案例 (假设 $\alpha_i = 1$):

x1	x2	y
1	1	-1
1	1.5	-1
1.5	1	-1
2	1.5	1
2	1	1
1.5	2	1

- i. Step 1: 计算 weighting factor, 因为 $\mathbf{w} = \sum_{i=1}^N \alpha_i y_i \mathbf{x}_i$, 在本题中, 我们假设 $\alpha_i = 1$, 因此 $\mathbf{w} = \sum_{i=1}^N y_i \mathbf{x}_i$ 。
 - $\sum_{y_i=1} \mathbf{x}_i = [2 + 2 + 1.5, 1.5 + 1 + 2] = [5.5, 4.5]$
 - $\sum_{y_i=-1} \mathbf{x}_i = [1 + 1 + 1.5, 1 + 1.5 + 1] = [3.5, 3.5]$

- $\mathbf{w} = [5.5 - 3.5, 4.5 - 3.5] = [2.0, 1.0]$

ii. Step 2: 计算偏置 w_0 , 我们的公式为 $w_0 = \frac{1}{N} \sum_i (y_i - \mathbf{w}^T \mathbf{x}_i)$

- No 1: $y_1 - \mathbf{w}^T \mathbf{x}_1 = -1 - (2 \times 1 + 1 \times 1) = -4$
- No 2: $y_2 - \mathbf{w}^T \mathbf{x}_2 = -1 - (2 \times 1 + 1 \times 1.5) = -4.5$
- No 3: $y_3 - \mathbf{w}^T \mathbf{x}_3 = -1 - (2 \times 1.5 + 1 \times 1) = -5$
- No 4: $y_4 - \mathbf{w}^T \mathbf{x}_4 = 1 - (2 \times 2 + 1 \times 1.5) = -4.5$
- No 5: $y_5 - \mathbf{w}^T \mathbf{x}_5 = 1 - (2 \times 2 + 1 \times 1) = -4$
- No 6: $y_6 - \mathbf{w}^T \mathbf{x}_6 = 1 - (2 \times 1.5 + 1 \times 2) = -4$
- 根据公式, 我们加和除 6

$$w_0 = \frac{-4 - 4.5 - 5 - 4.5 - 4 - 4}{6} = \frac{-26}{6} = \boxed{-4.33}$$

iii. 写出 $h(x) = 2x_1 + x_2 - 4.33$, 我们的规则为 $\hat{y} = \text{sign}(h(\mathbf{x}))$ 。

d. 案例 (给定 α_i)

x1	x2	y	α
1	1	-1	0
1	1.5	-1	0
1.5	1	-1	10
2	1.5	1	0
2	1	1	8
1.5	2	1	2

i. Step 1: 计算 weighting factor, 因为 $\mathbf{w} = \sum_{i=1}^N \alpha_i y_i \mathbf{x}_i$ 。我们只使用 supporting vectors 参与计算, 即 $\alpha_i > 0$ 的。

- 对于 $y = -1$, 因此 $10 \cdot (-1) \cdot (1.5, 1) = (-15, -10)$
- 对于 $y = 1$, 因此 $8 \cdot 1 \cdot (2, 1) = (16, 8)$ 和 $2 \cdot 1 \cdot (1.5, 2) = (3, 4)$
- $\mathbf{w} = (-15 + 16 + 3, -10 + 8 + 4) = (4, 2)$

ii. Step 2: 计算偏置 w_0 , 我们的公式为 $w_0 = \frac{1}{N} \sum_i (y_i - \mathbf{w}^T \mathbf{x}_i)$

- No 3: $y_3 - \mathbf{w}^T \mathbf{x}_3 = -1 - (4 \times 1.5 + 2 \times 1) = -9$
- No 5: $y_5 - \mathbf{w}^T \mathbf{x}_5 = 1 - (4 \times 2 + 2 \times 1) = -9$
- No 6: $y_6 - \mathbf{w}^T \mathbf{x}_6 = 1 - (4 \times 1.5 + 2 \times 2) = -9$
- 根据公式, 我们加和除 3

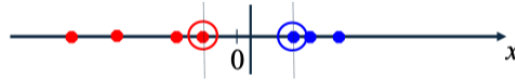
$$w_0 = \frac{-9 + (-9) + (-9)}{3} = \boxed{-9}$$

iii. 写出 $h(x) = 4x_1 + 2x_2 - 9$, 我们的规则为 $\hat{y} = \text{sign}(h(\mathbf{x}))$ 。

Non-Linearity

14. 当数据在原始空间中线性不可分时, 我们应该怎么办?

- a. 这是一个 linearly separable 的数据集，红点在左，蓝点在右。我们用一个简单的线性决策边界 ($x = 0$) 就可以分类。这正是 Perceptron 和 SVM 擅长的情况。



- b. 对于一个不 linearly separable 的数据集，红蓝点交错分布，无法通过一条直线划分。



- c. 解决方案: mapping data to a higher-dimensional space, 比如将 x 映射到 x^2 , 即引入一个新特征。

$$\Phi: \mathbf{x} \mapsto \phi(\mathbf{x})$$

表示一种从低维空间 (如二维) 到高维空间 (如三维) 的变换。 $\phi(\mathbf{x})$ 是对输入特征 $\mathbf{x}$ 的非线性变换 nonlinear transformation, 例如

$$\phi(x_1, x_2) = (x_1, x_2, x_1^2 + x_2^2)$$

很多看似无法线性分类的数据, 其实可以通过 Feature Mapping 到更高维后再做线性分类。

15. Some Kernels

- a. Kernels 公式 ($\mathbf{x}'$ 是训练集中的 support vector; $\mathbf{x}$ 是当前待分类的样本)
- Linear kernel: $K(\mathbf{x}, \mathbf{x}') = \mathbf{x}^T \mathbf{x}'$ (就是我们上述的普通 SVM 方法)
 - Quadratic Kernel: $K(\mathbf{x}, \mathbf{x}') = (\mathbf{x}^T \mathbf{x}' + 1)^2$
 - RBF Kernel: $K(\mathbf{x}, \mathbf{x}') = \exp(-\gamma \|\mathbf{x} - \mathbf{x}'\|^2)$
 - $\gamma > 0$ 是一个可调参数, 常设为 $\gamma = 1/2\sigma^2$
- b. 对偶形式与 Kernels

$$h(\mathbf{x}) = \sum_i \alpha_i y_i K(\mathbf{x}_i, \mathbf{x}) + w_0$$

$$w_0 = y_s - \sum_{i=1}^N \alpha_i y_i K(\mathbf{x}_i, \mathbf{x}_s)$$

我们使用该公式套入之前的 Kernel 公式, 就可以得到

- Linear Kernel: $h(\mathbf{x}) = \sum_i \alpha_i y_i \mathbf{x}_i^T \mathbf{x}' + w_0$
 - 在原始形式中, 我们 $h(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + w_0$, 我们通过优化得到 $\mathbf{w}$, 例如在硬间隔 SVM 中是 $\mathbf{w} = \sum_{i=1}^N \alpha_i y_i \mathbf{x}_i$, 因此

$$h(\mathbf{x}) = \left(\sum_i \alpha_i y_i \mathbf{x}_i \right)^T \mathbf{x} + w_0 = \sum_i \alpha_i y_i \mathbf{x}_i^T \mathbf{x} + w_0$$

- 所以我们的原始形式和对偶形式+Linear Kernel 完全等价。

- ii. Quadratic Kernel: $h(\mathbf{x}) = \sum_i \alpha_i y_i (x^T x' + 1)^2 + w_0$
 iii. RBF Kernal: $h(\mathbf{x}) = \sum_i \alpha_i y_i \exp(-\gamma \|\mathbf{x} - \mathbf{x}'\|^2) + w_0$
 c. 案例（使用对偶形式+ Quadratic Kernel）:

x1	x2	y	α
1	1	-1	0
1	1.5	-1	0
1.5	1	-1	10
2	1.5	1	0
2	1	1	8
1.5	2	1	2

- i. Step 1: 我们的公式为 $h(\mathbf{x}) = \sum_i \alpha_i y_i (x^T x' + 1)^2 + w_0$, 我们现在先算 $y_i (x^T x' + 1)^2$

- Example 1: $-1 \times ((1,1) \cdot \mathbf{x} + 1)^2 = -(x_1 + x_2 + 1)^2$
- Example 2: $-(x_1 + 1.5x_2 + 1)^2$
- Example 3: $-(1.5x_1 + x_2 + 1)^2$
- Example 4: $+(2x_1 + 1.5x_2 + 1)^2$
- Example 5: $+(2x_1 + x_2 + 1)^2$
- Example 6: $+(1.5x_1 + 2x_2 + 1)^2$

- ii. Step 2: 写出公式, 除了 w_0 的部分, 这里我们假设 $\alpha_i = 1$

$$\begin{aligned}
 h(\mathbf{x}) = & -(x_1 + x_2 + 1)^2 \\
 & -(x_1 + 1.5x_2 + 1)^2 \\
 & -(1.5x_1 + x_2 + 1)^2 \\
 & +(2x_1 + 1.5x_2 + 1)^2 \\
 & +(2x_1 + x_2 + 1)^2 \\
 & +(1.5x_1 + 2x_2 + 1)^2 \\
 & +w_0
 \end{aligned}$$

- iii. Step 3: 计算偏置 w_0 。我们的公式为 $w_0 = y_s - \sum_{i=1}^N \alpha_i y_i K(\mathbf{x}_i, \mathbf{x}_s)$, 因此我们要随机选择一个 y_s , 在这里我们选择 y_3 , 并且我们假设 $\alpha_i = 1$ 。

i	$\mathbf{x}_i$	y_i	$K(\mathbf{x}_i, \mathbf{x}_3)$
1	(1, 1)	-1	$(2.5 + 1)^2 = 12.25$
2	(1, 1.5)	-1	$(3 + 1)^2 = 16$
3	(1.5, 1)	-1	$(3.25 + 1)^2 = 18.0625$
4	(2, 1.5)	1	$(4.5 + 1)^2 = 30.25$
5	(2, 1)	1	$(4 + 1)^2 = 25$
6	(1.5, 2)	1	$(4.25 + 1)^2 = 27.5625$

然后我们加权求和

$$\sum_{i=1}^6 \alpha_i y_i K(\mathbf{x}_i, \mathbf{x}_3)$$

$$= (-1)(12.25) + (-1)(16) + (-1)(18.0625) + (1)(30.25) + (1)(25) + (1)(27.5625) = \boxed{36.5}$$

然后我们代入公式计算偏置

$$w_0 = y_3 - \sum_{i=1}^6 \alpha_i y_i K(\mathbf{x}_i, \mathbf{x}_3) = -1 - 36.5 = \boxed{-37.5}$$

16. Why Kernels?

- a. 能不能不用 kernel，自己手动扩展特征？
 - i. 是的，原则上可以：你可以把输入的每个维度手动扩展成高维特征（比如加平方项、交叉项等），这就等价于用了 quadratic kernel。
 - ii. 不需要修改算法：因为本质上还是个线性模型，只是特征换了。
 - iii. 但有问题：特征维度可能变得非常高甚至无限（比如 RBF kernel 对应的是无限维空间）。某些 kernel（特别是 RBF）根本没法显式地表示为特征组合。
- b. 那 kernel 有什么优势？
 - i. 核函数允许我们不用真的把特征变成高维，就能直接计算在高维空间的“内积”。
 - ii. 好处：节省计算和存储空间，速度更快。
 - iii. 代价：要使用“对偶形式”，即：在预测时，你需要对每一个训练样本和新样本之间都计算一次核函数（也就是相似度）。

Clustering

17. 什么是 Clustering System?

- a. 是一种 unsupervised learning，意思是你有 features，但是没有 labels。系统需要自动发现数据中的结构或模式。
- b. Clustering 的用途：分组电子邮件或搜索结果、检测异常程序执行。

18. Clustering 算法流程

- a. 随机选择 K 个数据点作为初始簇中心（means）。
- b. 迭代两步（直到收敛）：
 - i. Assignment Step（分配）：把每个数据点分配给离它最近的簇中心。
 - ii. Update Step（更新）：对每个簇，计算其成员点的平均值作为新的簇中心。
- c. 停止条件：当所有数据点的归属簇不再改变时（即收敛），算法终止。

19. K-Means 是 non-deterministic 的

- a. K-Means 算法每次运行的结果可能不同。
- b. 这是因为它依赖初始簇中心 (initial means) 的选择。
- c. 如果初始值 (means) 选得不好, 可能导致:
 - i. 局部最优 (suboptimal clustering): K-Means 不是全局最优的保证算法。即使数据结构明显存在良好的聚类划分, 也可能因为初始中心选择不当, 导致算法卡在局部最优。
 - ii. 簇中心都聚在一边, 导致某些簇永远没有点被分配

20. 常见问题

- a. Will K-means converge? K-Means 总是会收敛 (每次迭代都会减少总距离), 但: 不一定收敛到全局最优, 常常只到达局部最优。
- b. Will it always find the true patterns in the data? 如果模式非常清晰 (如球状、距离分明), K-Means 可能有效。
- c. Will it find something interesting? 有时结果有意义, 有时完全不符合实际问题——因为它只是基于距离最小化, 没有语义理解。